



(54) **LOW-PRECISION FLOATING-POINT DATAPATH IN A COMPUTER PROCESSOR**

(71) Applicant: **NVIDIA Corp.**, Santa Clara, CA (US)

(72) Inventors: **Rangharajan Venkatesan**, Sunnyvale, CA (US); **Reena Elangovan**, West Lafayette, IN (US); **Charbel Sakr**, San Jose, CA (US); **Brucek Kurdo Khailany**, Rollingwood, TX (US); **Ming Y Siu**, Santa Clara, CA (US); **Ilyas Elkin**, Sunnyvale, CA (US); **Brent Ralph Boswell**, Aloha, OR (US)

(73) Assignee: **NVIDIA Corp.**, Santa Clara, CA (US)

(21) Appl. No.: **18/484,790**

(22) Filed: **Oct. 11, 2023**

Related U.S. Application Data

(60) Provisional application No. 63/383,626, filed on Nov. 14, 2022.

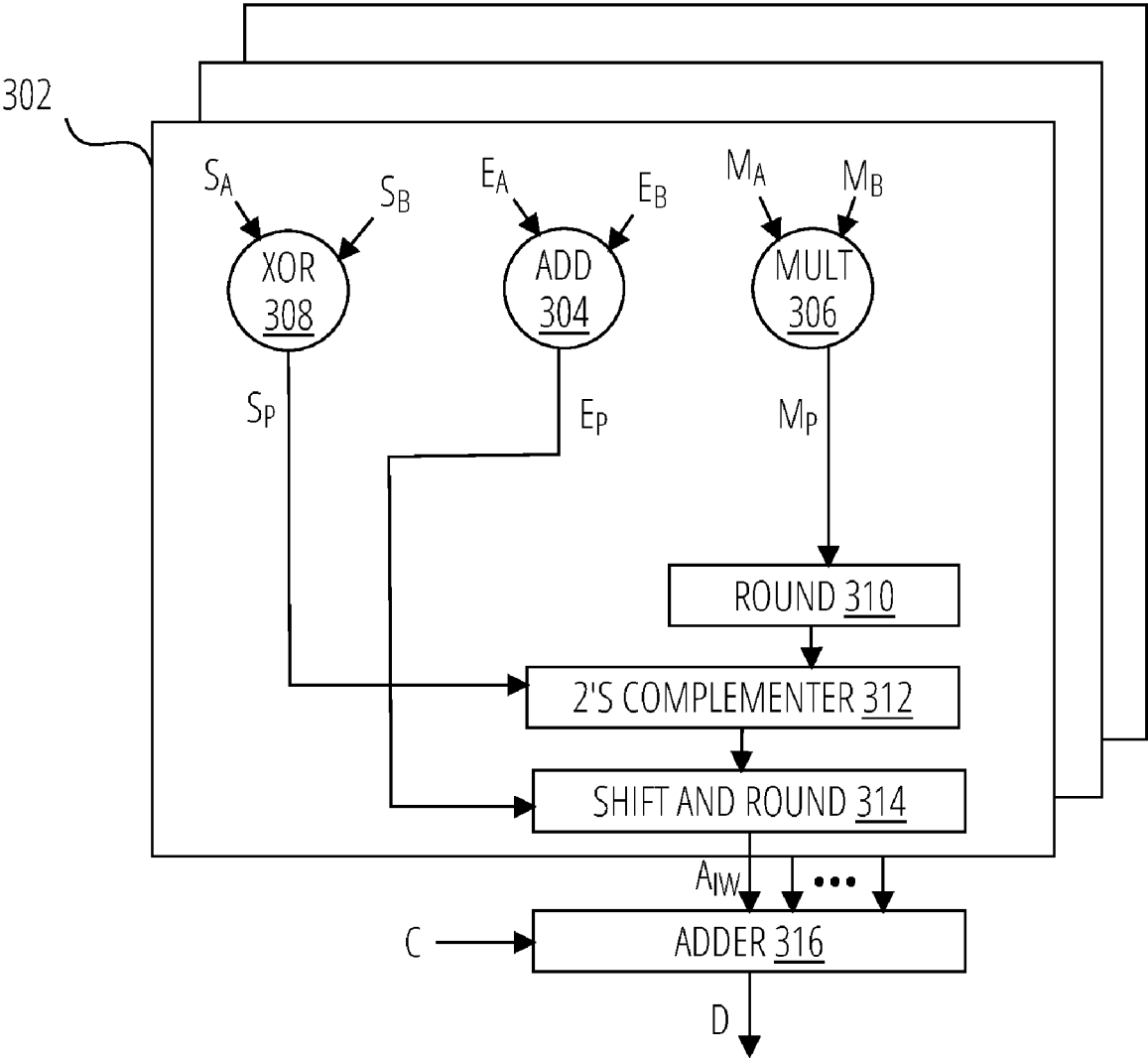
Publication Classification

(51) **Int. Cl.**
G06F 7/487 (2006.01)
G06F 7/499 (2006.01)

(52) **U.S. Cl.**
CPC **G06F 7/4876** (2013.01); **G06F 7/49915** (2013.01)

(57) **ABSTRACT**

Mechanisms to exploit the inherent resiliency of deep learning inference workloads to improve the energy efficiency of computer processors such as graphics processing units with these workloads. The mechanisms provide energy-accuracy tradeoffs in the computation of deep learning inference calculations via energy-efficient floating point data path micro-architectures with integer accumulation, and enhanced mechanisms for per-vector scaled quantization (VS-Quant) of floating-point arguments.



	E4M3	E5M2
Exponent bias	7	15
Infinities	N/A	$S.11111.00_2$
NaN	$S.1111.111_2$	$S.11111.\{01, 10, 11\}_2$
Zeros	$S.0000.000_2$	$S.00000.00_2$
Max normal	$S.1111.110_2 = 1.75 * 2^8 = 448$	$S.11110.11_2 = 1.75 * 2^{15} = 57,344$
Min normal	$S.0001.000_2 = 2^{-6}$	$S.00001.00_2 = 2^{-14}$
Max subnorm	$S.0000.111_2 = 0.875 * 2^{-6}$	$S.00000.11_2 = 0.75 * 2^{-14}$
Min subnorm	$S.0000.001_2 = 2^{-9}$	$S.00000.01_2 = 2^{-16}$

FIG. 1

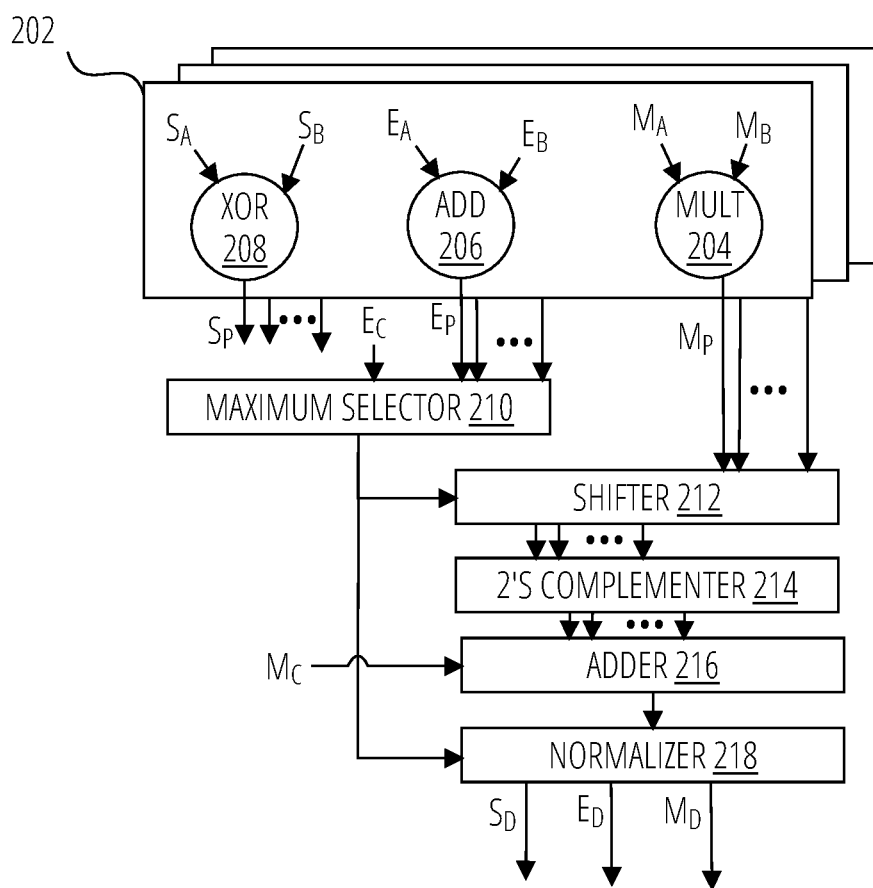


FIG. 2

PRIOR ART

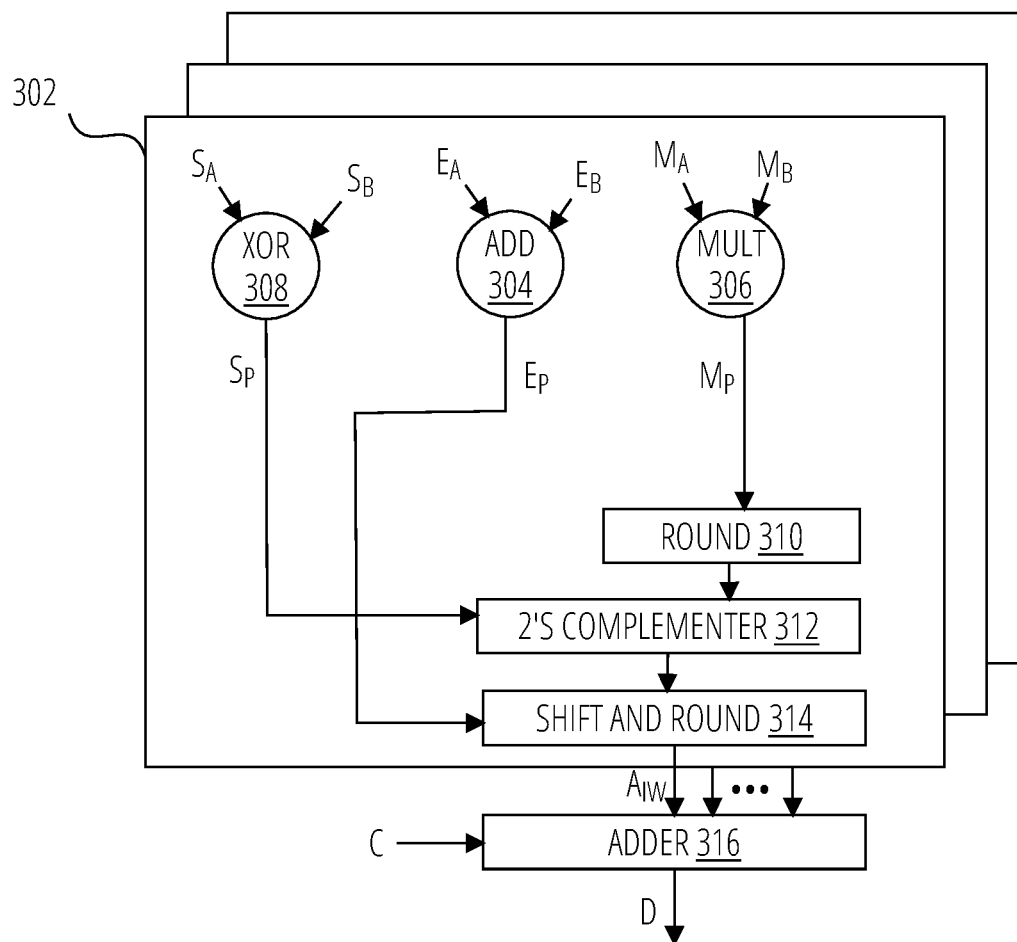


FIG. 3

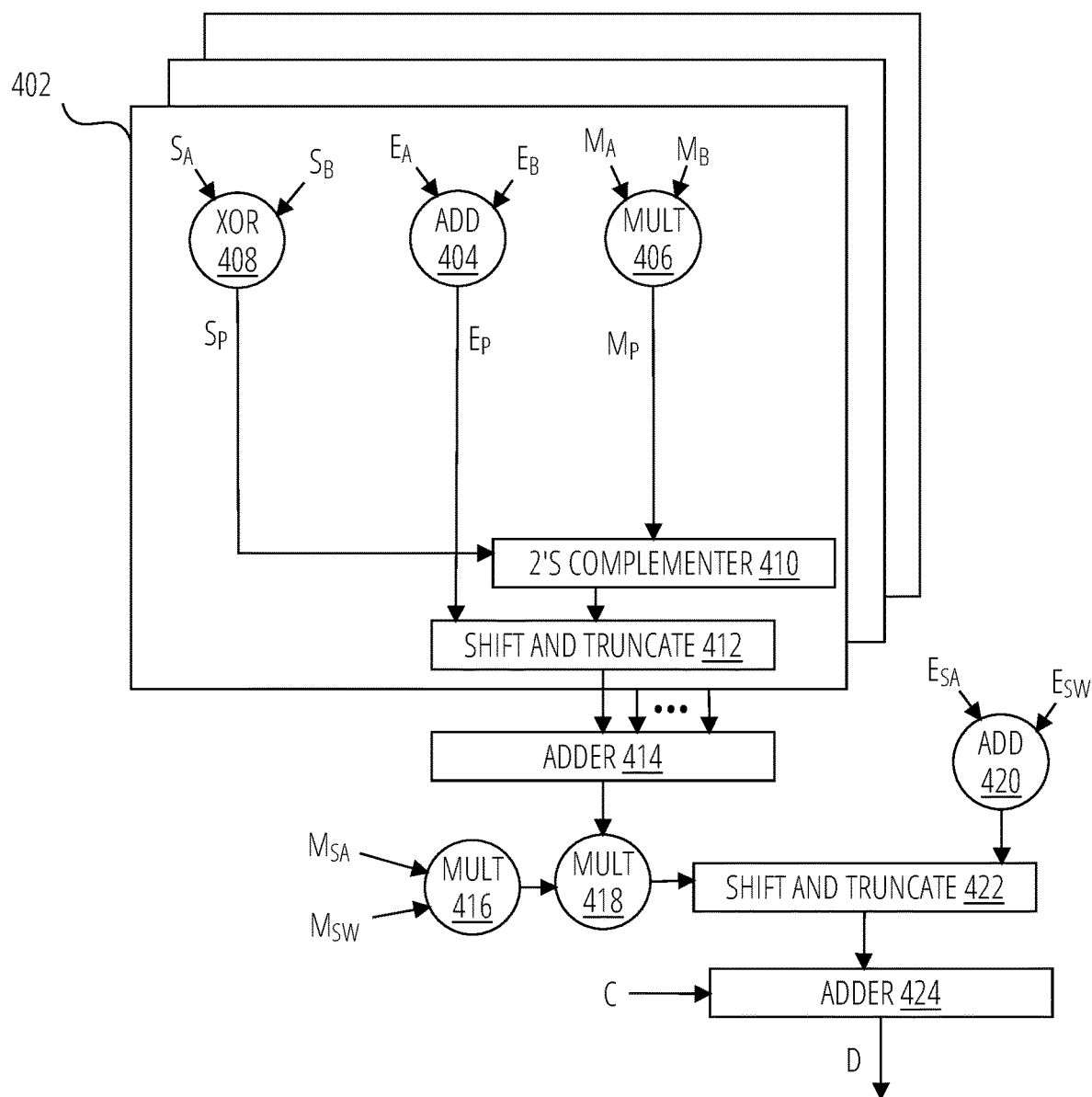


FIG. 4A

fp4e3m0				fp4e2m1				fp4e1m2				fp4e0m3			
sign	exp (bias 3)	mantissa	decimal	sign	exp (bias 1)	mantissa	decimal	sign	exp (bias 0)	mantissa	decimal	sign	exp (bias -)	mantissa	decimal
0	000	-	0.00	0	00	0	0.00	0	0	00	0.00	0	-	000	0
0	001	-	0.25	0	00	1	0.50	0	0	01	0.50	0	-	001	1
0	010	-	0.50	0	01	0	1.00	0	0	10	1.00	0	-	010	2
0	011	-	1.00	0	01	1	1.50	0	0	11	1.50	0	-	011	3
0	100	-	2.00	0	10	0	2.00	0	1	00	2.00	0	-	100	4
0	101	-	4.00	0	10	1	3.00	0	1	01	3.00	0	-	101	5
0	110	-	8.00	0	11	0	4.00	0	1	10	3.00	0	-	110	6
0	111	-	16.00	0	11	1	6.00	0	1	11	3.50	0	-	111	7
1	000	-	0.00	1	00	0	0.00	1	0	00	0.00	1	-	000	0
1	001	-	-0.25	1	00	1	-0.50	1	0	01	-0.50	1	-	001	-1
1	010	-	-0.50	1	01	0	-1.00	1	0	10	-1.00	1	-	010	-2
1	011	-	-1.00	1	01	1	-1.50	1	0	11	-1.50	1	-	011	-3
1	100	-	-2.00	1	10	0	-2.00	1	1	00	-2.00	1	-	100	-4
1	101	-	-4.00	1	10	1	-3.00	1	1	01	-2.50	1	-	101	-5
1	110	-	-8.00	1	11	0	-4.00	1	1	10	-3.00	1	-	110	-6
1	111	-	-16.00	1	11	1	-5.00	1	1	11	-3.50	1	-	111	-7

ufp4e4m0				ufp4e3m1				ufp4e2m2				ufp4e1m3			
sign	exp (bias 7)	mantissa	decimal	sign	exp (bias 3)	mantissa	decimal	sign	exp (bias 1)	mantissa	decimal	sign	exp (bias 0)	mantissa	decimal
-	0000	-	0.000000	-	000	0	0.000	-	00	00	0.00	-	0	000	0.00
-	0001	-	0.015625	-	000	1	0.125	-	00	01	0.125	-	0	001	0.25
-	0010	-	0.031250	-	001	0	0.250	-	00	10	0.50	-	0	010	0.50
-	0011	-	0.062500	-	001	1	0.375	-	00	11	0.75	-	0	011	0.75
-	0100	-	0.125000	-	010	0	0.500	-	01	00	1.00	-	0	100	1.00
-	0101	-	0.250000	-	010	1	0.750	-	01	01	1.25	-	0	101	1.25
-	0110	-	0.500000	-	011	0	1.000	-	01	10	1.50	-	0	110	1.50
-	0111	-	1.000000	-	011	1	1.500	-	01	11	1.75	-	0	111	1.75
-	1000	-	2.000000	-	100	0	2.000	-	10	00	2.00	-	1	000	2.00
-	1001	-	4.000000	-	100	1	3.000	-	10	01	2.50	-	1	001	2.25
-	1010	-	8.000000	-	101	0	4.000	-	10	10	3.00	-	1	010	2.50
-	1011	-	16.000000	-	101	1	6.000	-	10	11	3.50	-	1	011	2.75
-	1100	-	32.000000	-	110	0	8.000	-	11	00	4.00	-	1	100	3.00
-	1101	-	64.000000	-	110	1	12.000	-	11	01	5.00	-	1	101	3.25
-	1110	-	128.000000	-	111	0	16.000	-	11	10	6.00	-	1	110	3.50
-	1111	-	256.000000	-	111	1	24.000	-	11	11	7.00	-	1	111	3.75

FIG. 4B

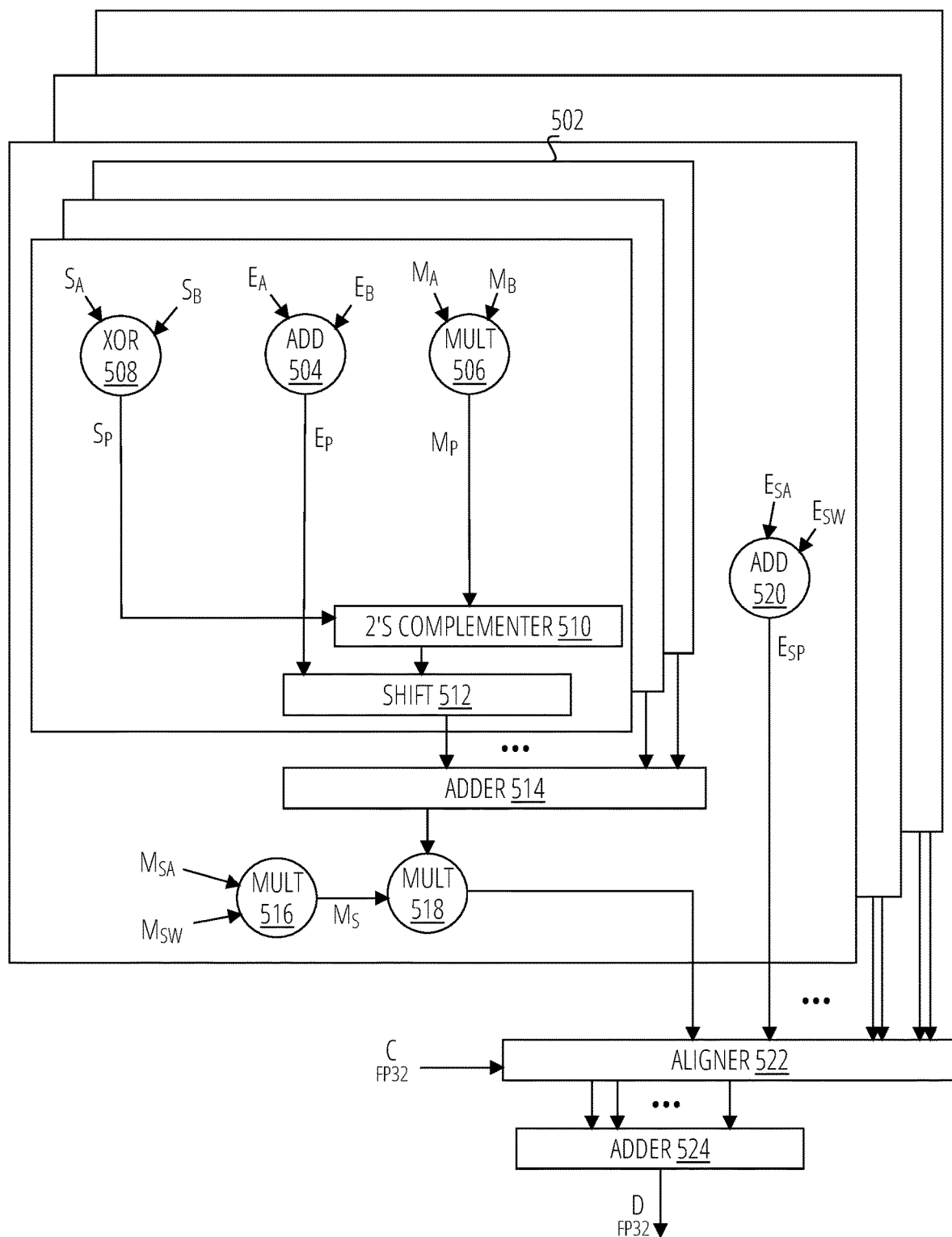


FIG. 5

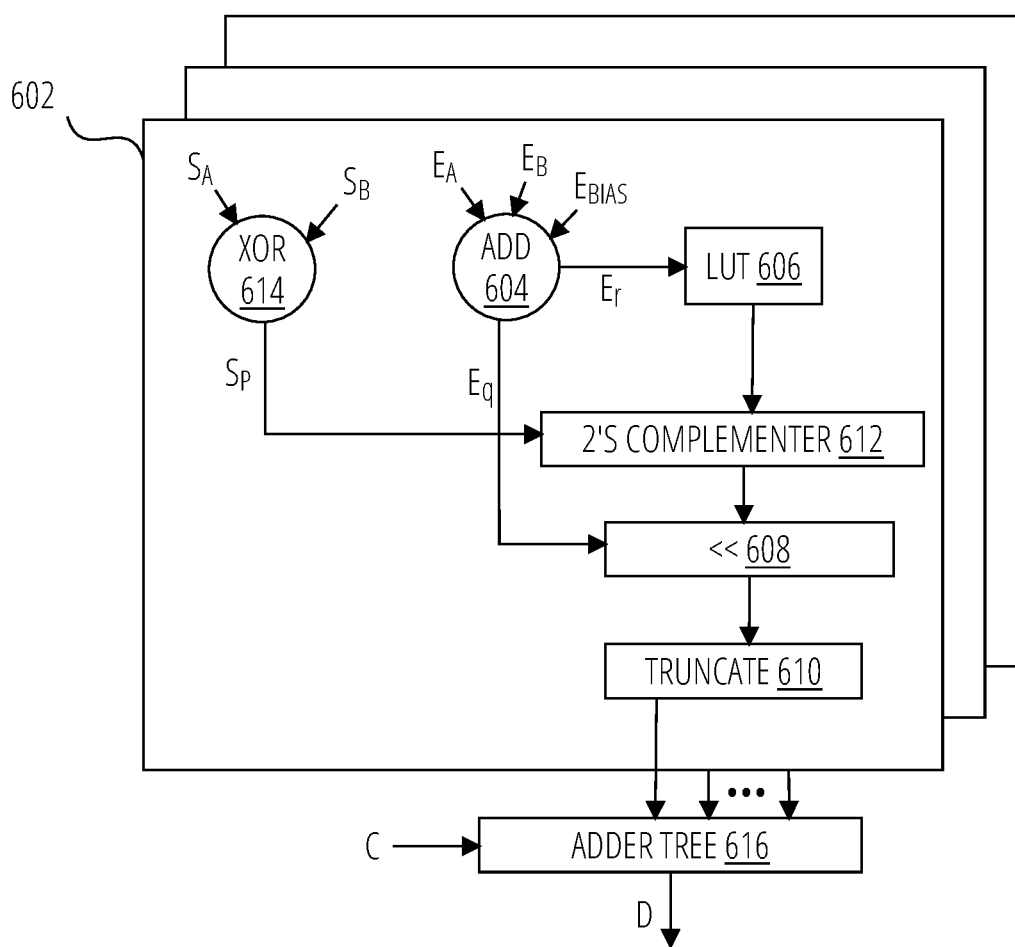


FIG. 6A

	E3.0/LOG4
Exponent Bias	3
Inf/NaN	N/A
Zeros	S000
Max. Normal	16
Min. Normal	0.25
Max. subnorm	0
Min. subnorm	0

FIG. 6B

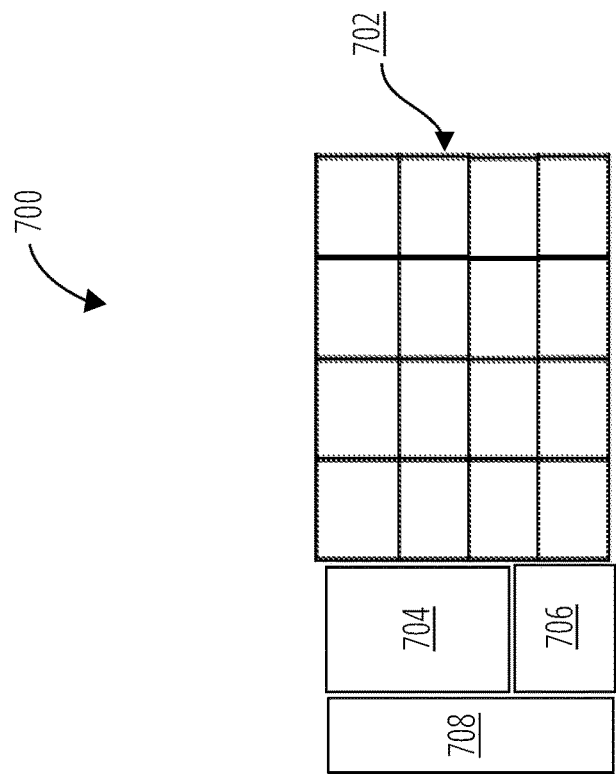


FIG. 7

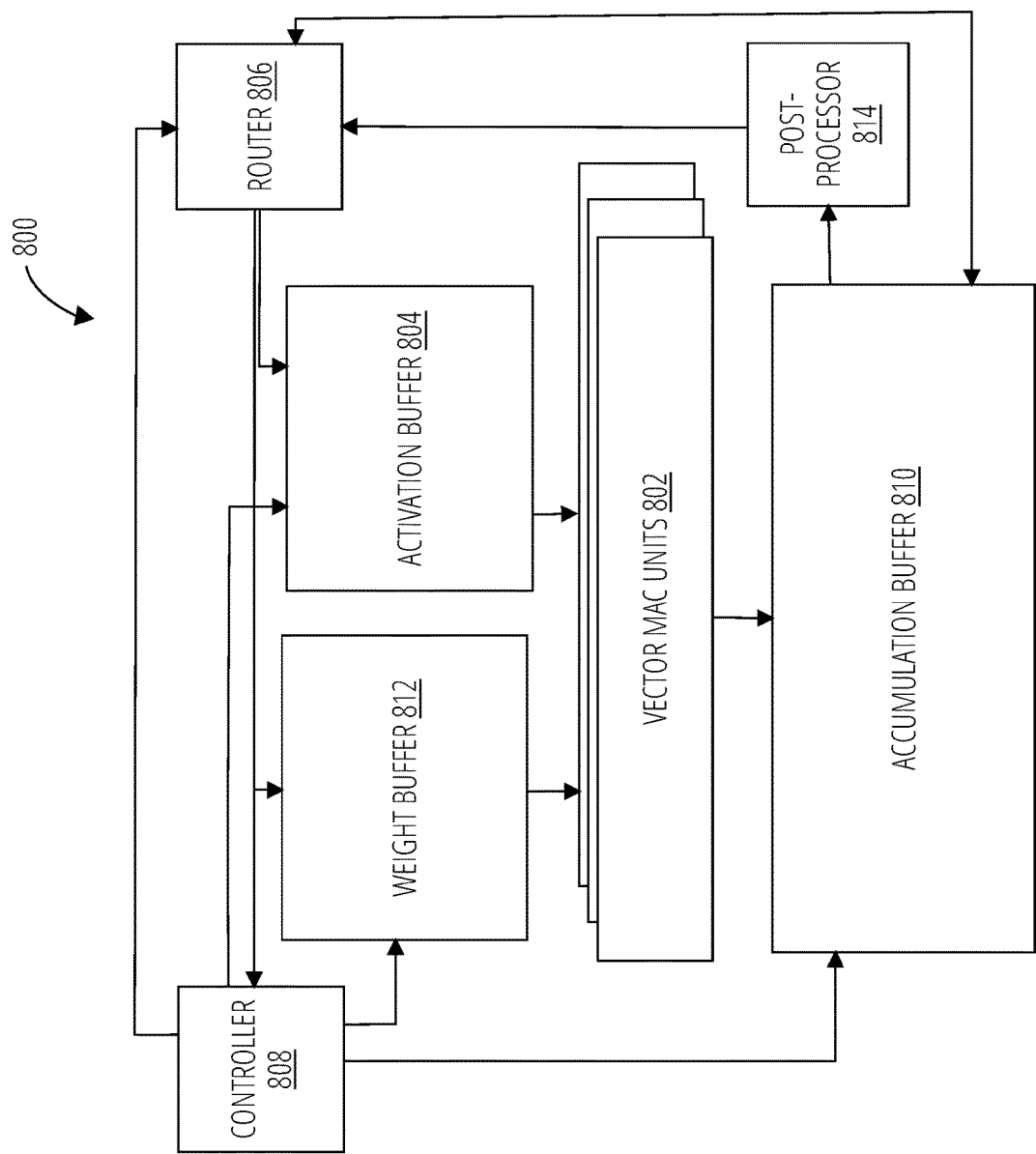


FIG. 8

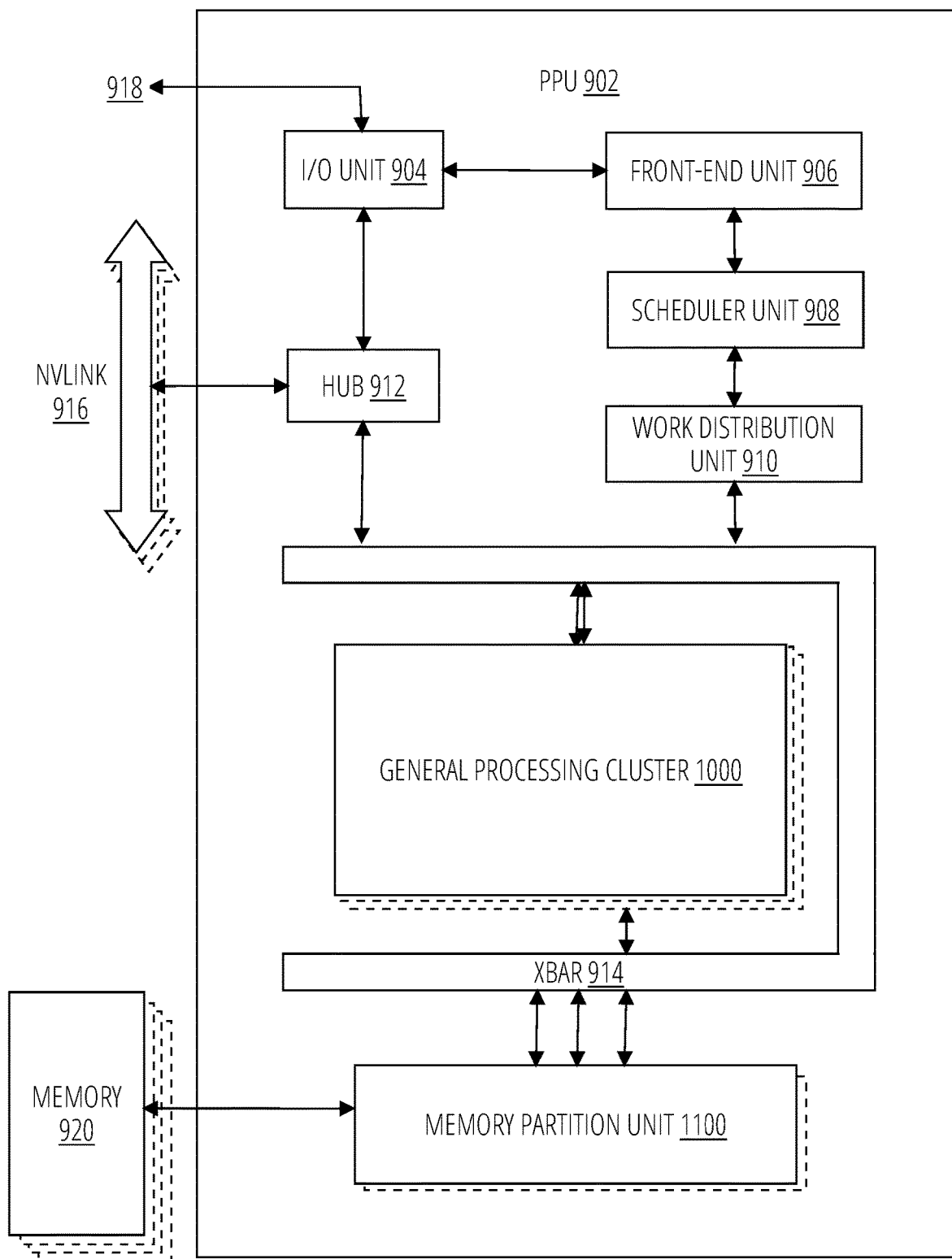


FIG. 9

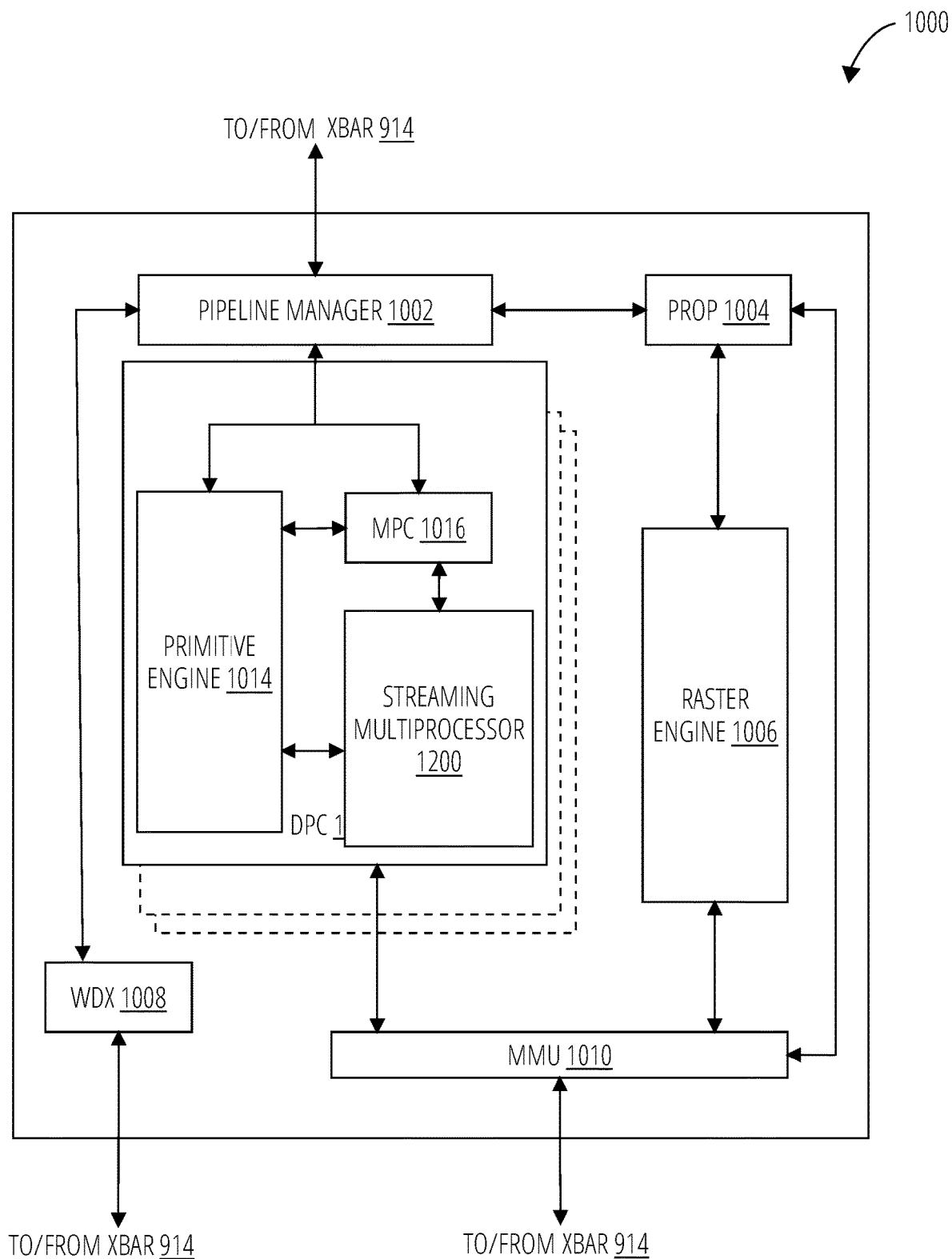


FIG. 10

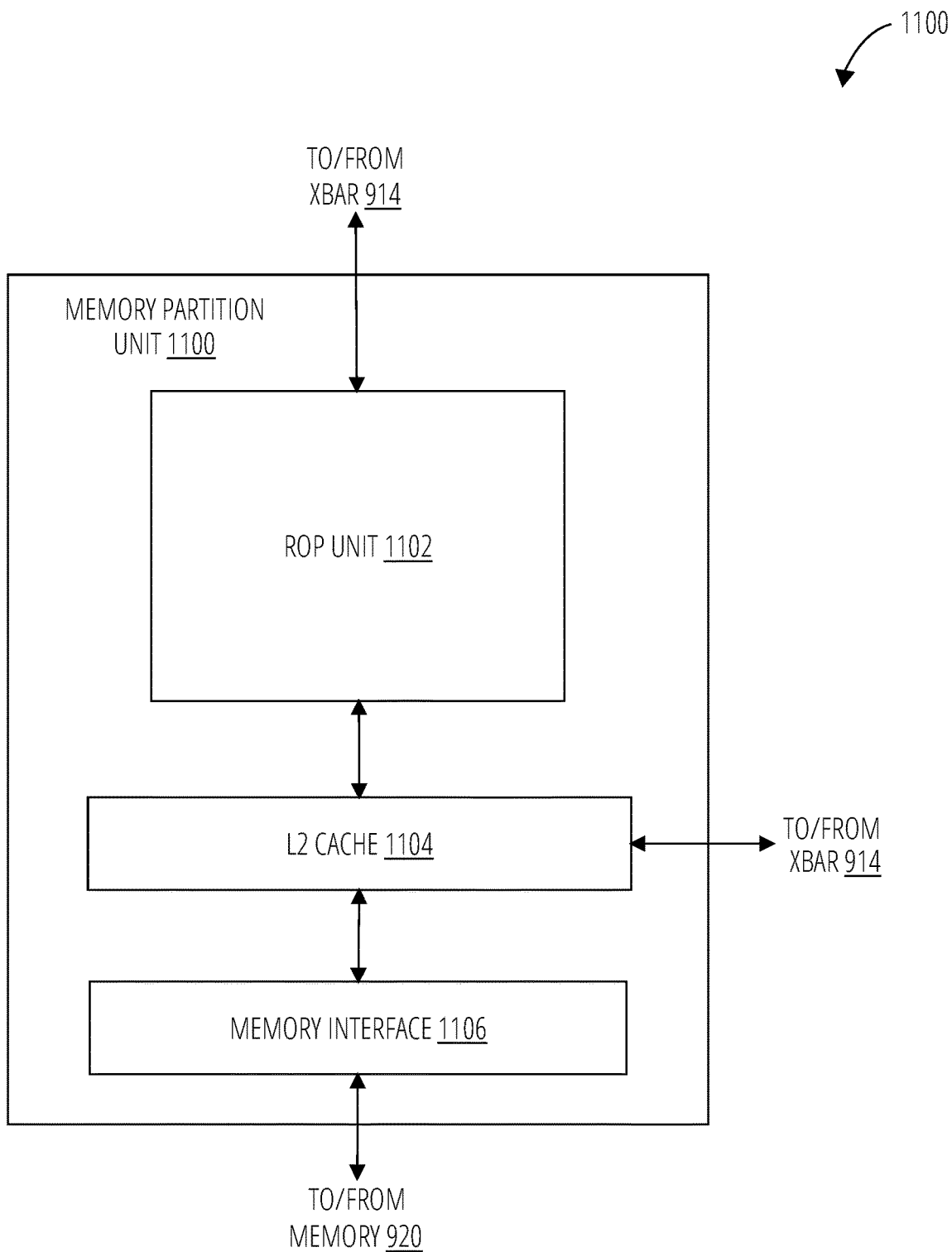


FIG. 11

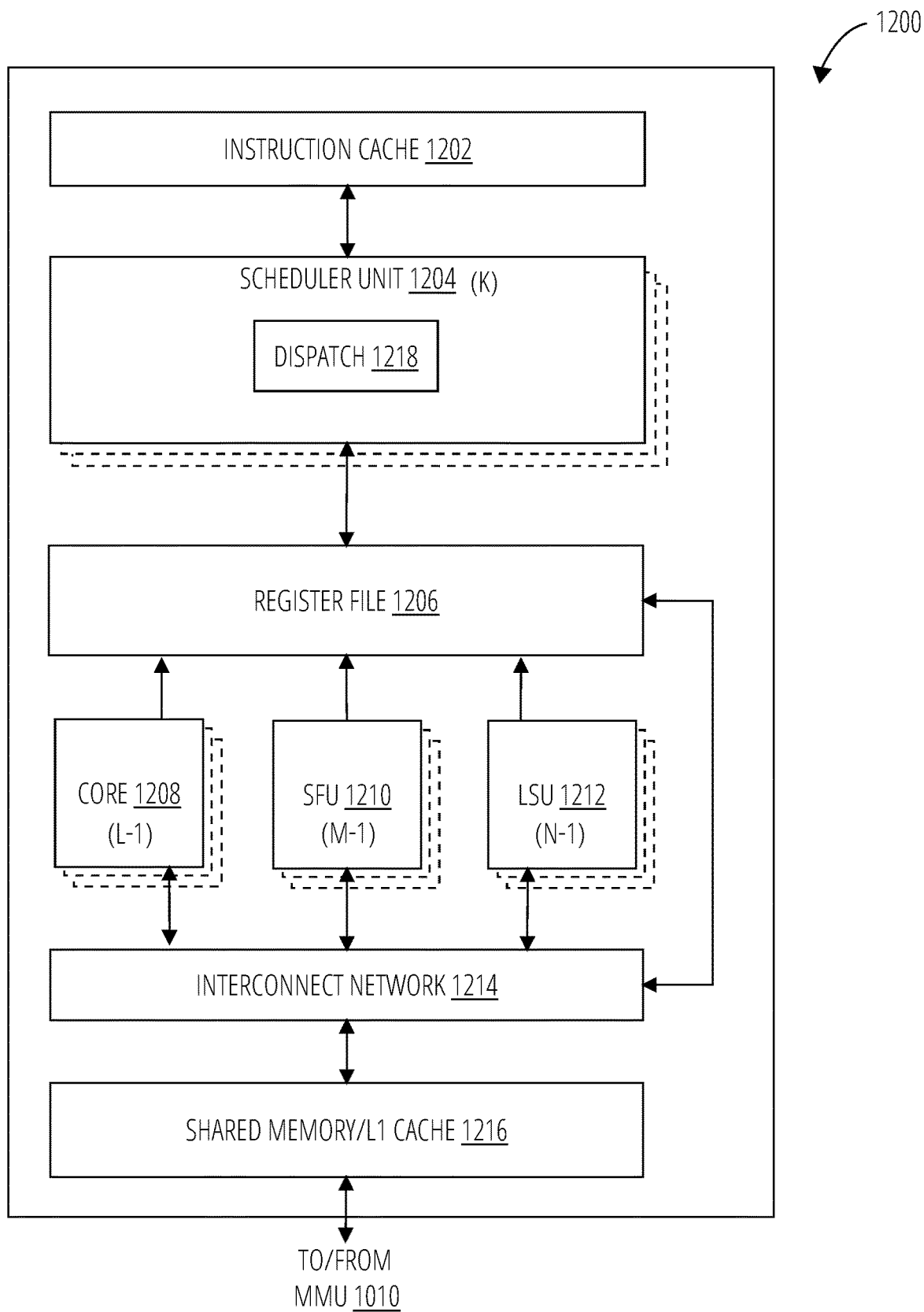


FIG. 12

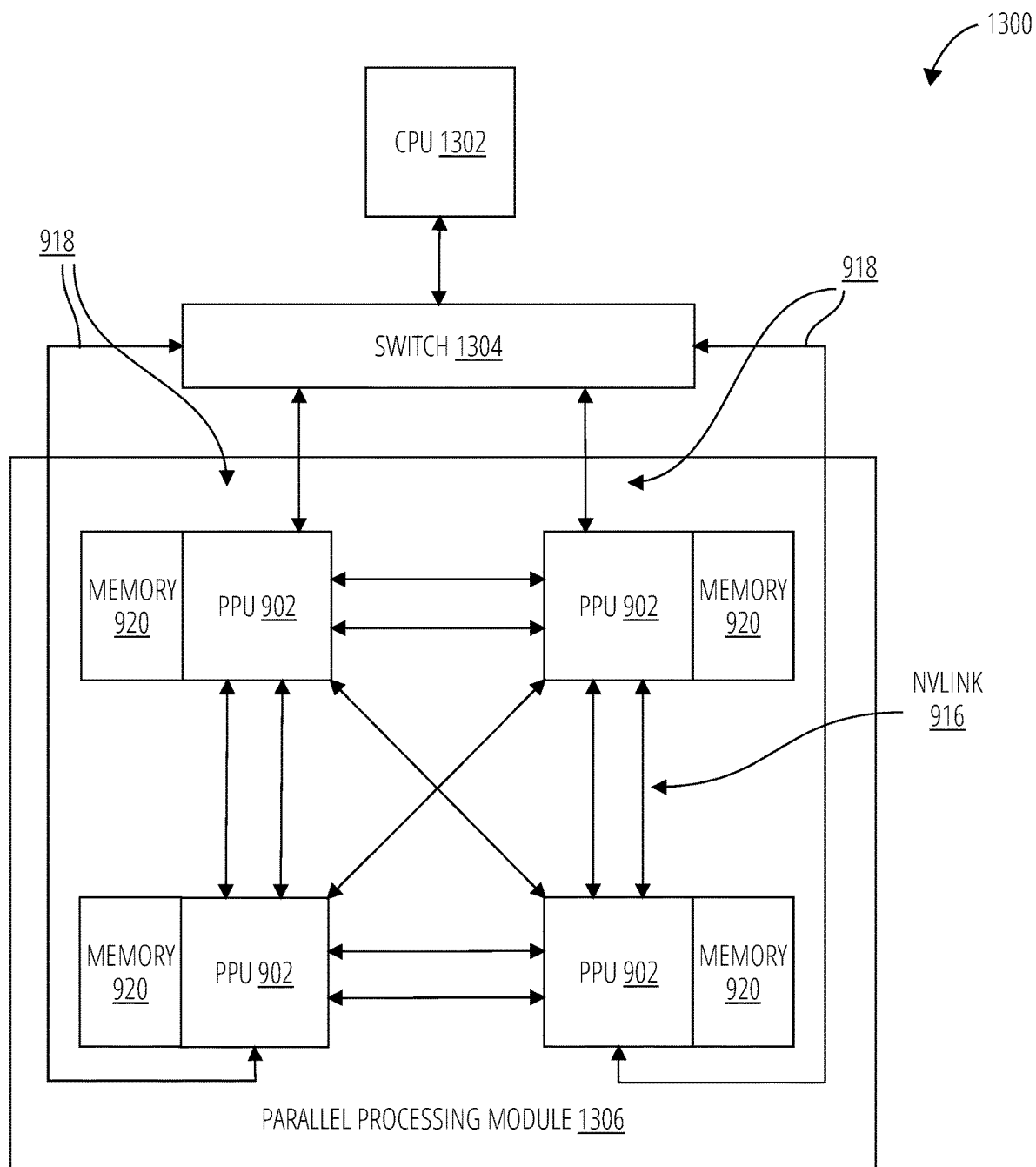


FIG. 13

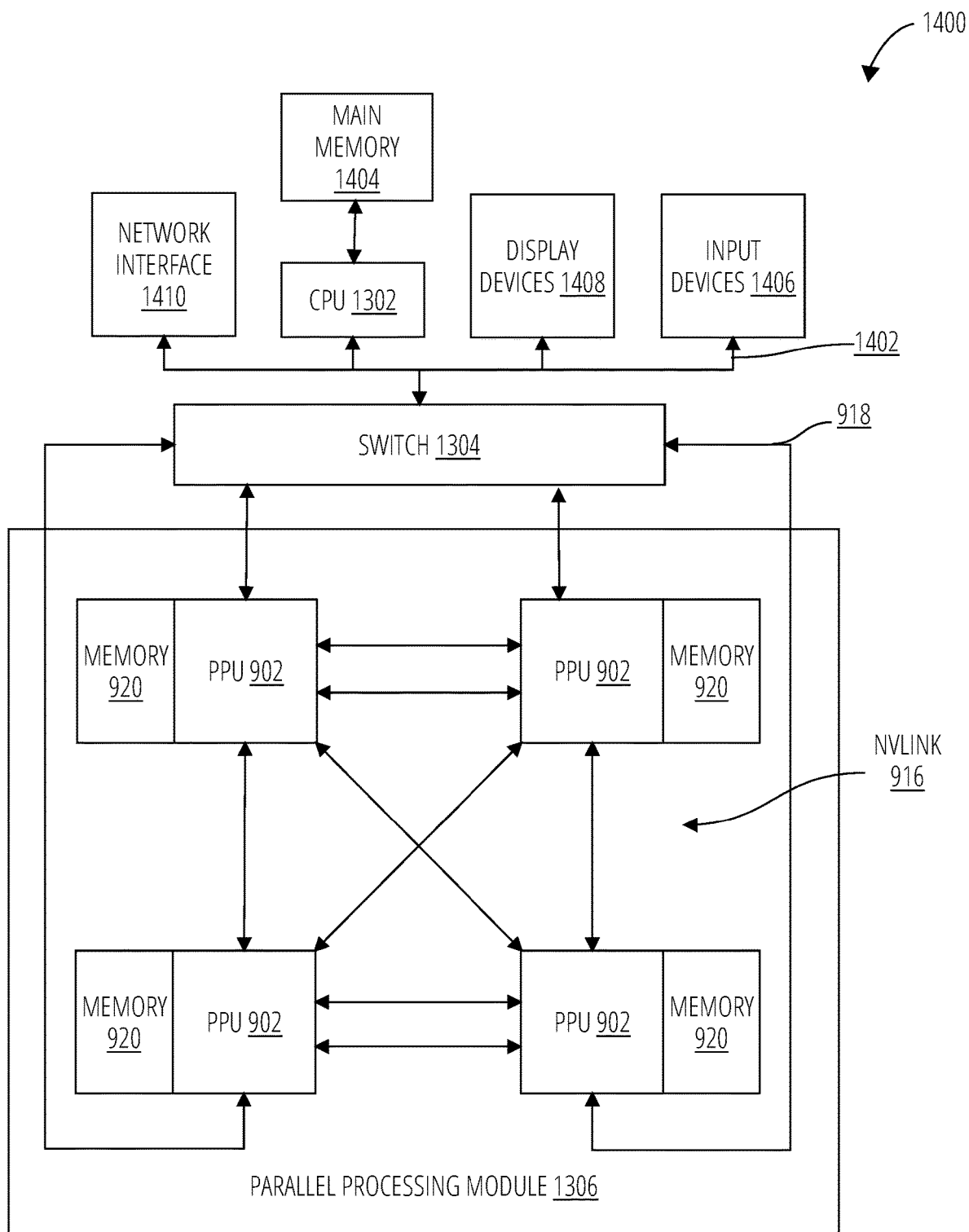
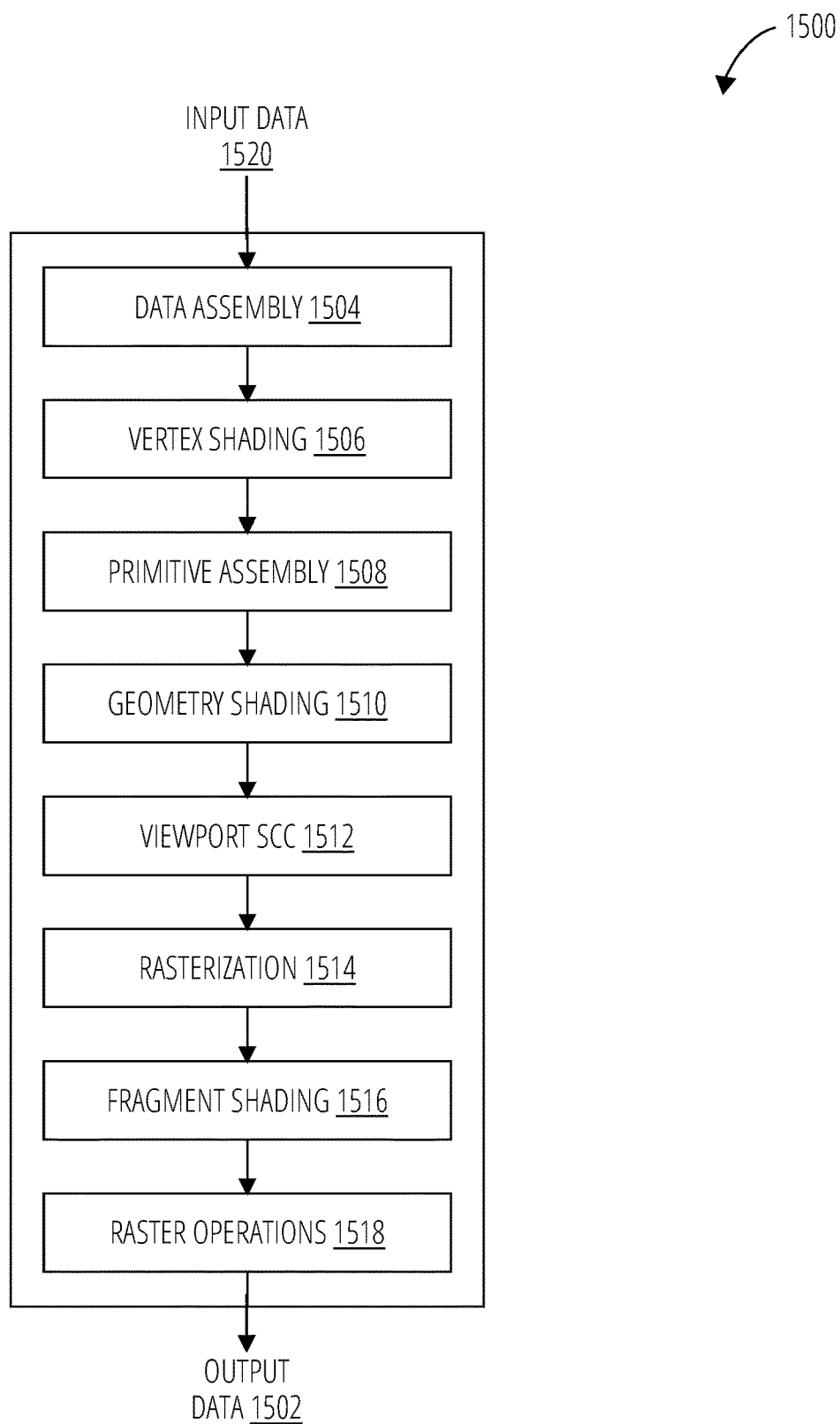


FIG. 14

**FIG. 15**

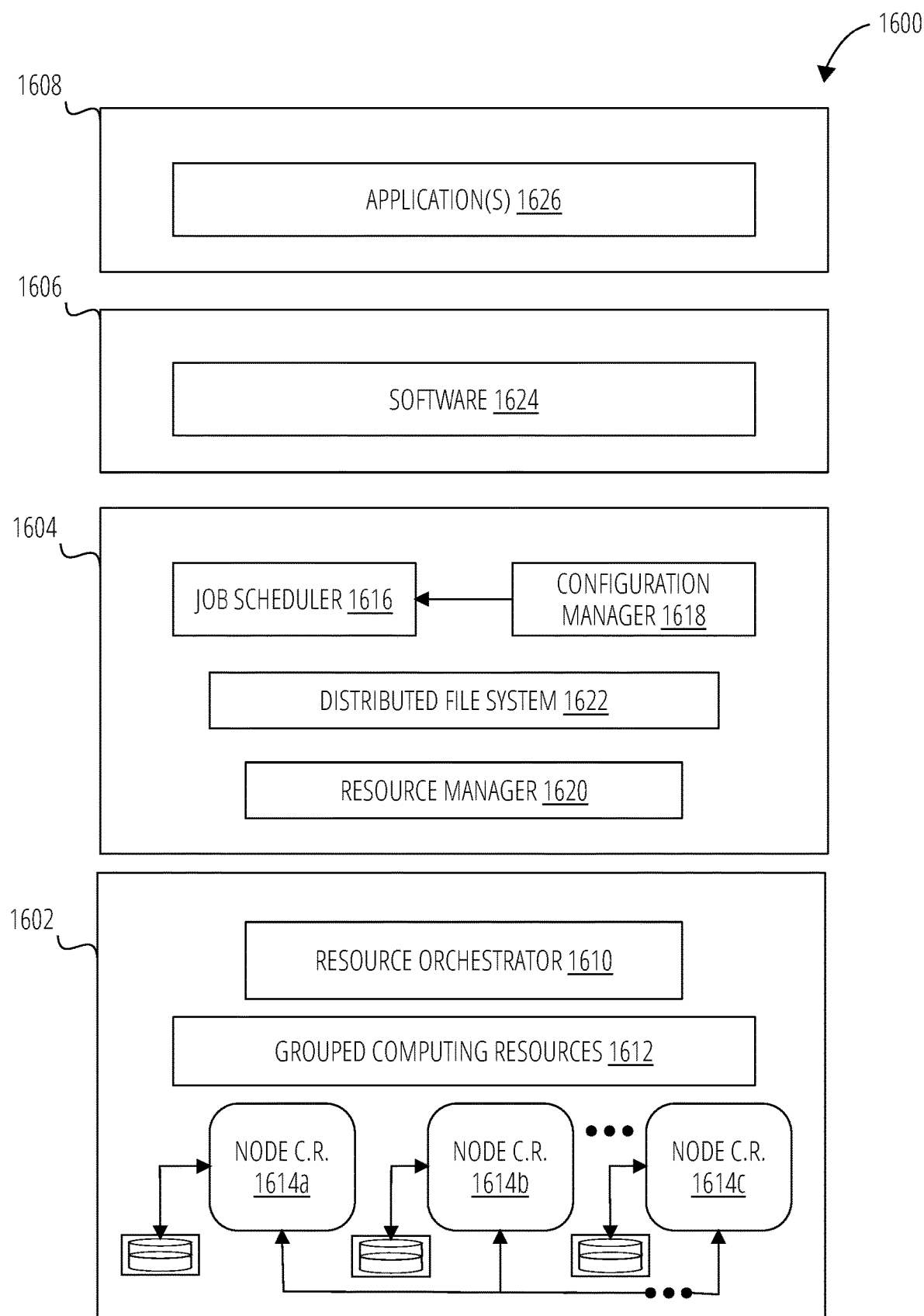


FIG. 16

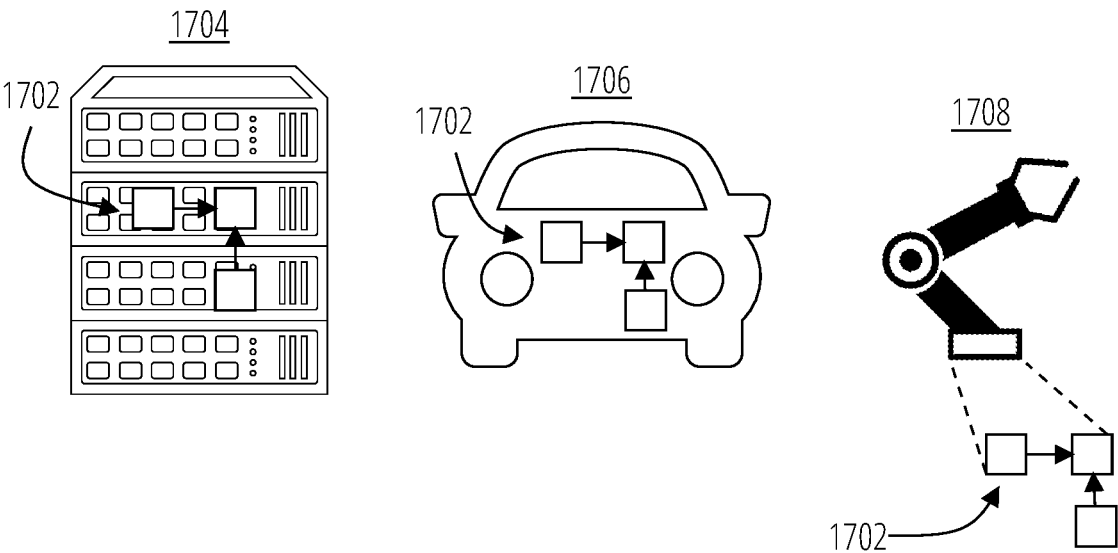


FIG. 17

LOW-PRECISION FLOATING-POINT DATAPATH IN A COMPUTER PROCESSOR

CROSS REFERENCE TO RELATED APPLICATIONS

[0001] This application claims priority and benefit under 35 USC 119(e) to U.S. application Ser. No. 63/383,626, “Low-Precision Floating-point Datapath In A Computer Processor”, filed on Nov. 14, 2022, the contents of which are incorporated herein by reference in their entirety.

BACKGROUND

[0002] Data paths contribute a significant fraction to the total energy consumption of graphics processing units (GPUs) for deep learning workloads. One mechanism to improve the energy-efficiency of data paths is utilization of lower precision and alternative data formats (e.g., Integer e.g., INT8, Floating-point, Log, VS-Quant). One such mechanism is utilization of 8-bit floating-point formats (FP8) that provide higher accuracy than the eight bit integer (INT8) formats. However, the higher accuracy of FP8 comes at the cost of lower energy efficiency than Int8.

BRIEF DESCRIPTION OF THE SEVERAL VIEWS OF THE DRAWINGS

[0003] To easily identify the discussion of any particular element or act, the most significant digit or digits in a reference number refer to the figure number in which that element is first introduced.

[0004] FIG. 1 depicts exemplary FP8 data formats.

[0005] FIG. 2 depicts a micro-architecture of a data path used in conventional vector multiply units 202 that operate on FP8 formatted operands.

[0006] FIG. 3 depicts an embodiment micro-architecture for the data path of vector multiply unit 302 (MAC).

[0007] FIG. 4A depicts an embodiment to further lower precision in the data path of vector multiply units 402 utilizing VS-Quant FP4 (four bit floating point) data formats.

[0008] FIG. 4B depicts exemplary FP4 data formats.

[0009] FIG. 5 depicts a data path in which INT accumulation is utilized to generate partial dot-products that are then accumulated in an FP32 format.

[0010] FIG. 6A depicts a data path of multiply-accumulate units 602 that operate on a VS-Quant LOG4 data format.

[0011] FIG. 6B depicts an E3.0/LOG4 data format.

[0012] FIG. 7 depicts a neural network processor 700 implemented on a single chip in accordance with one embodiment.

[0013] FIG. 8 depicts a local processing element 800 in accordance with one embodiment.

[0014] FIG. 9 depicts a parallel processing unit 902 in accordance with one embodiment.

[0015] FIG. 10 depicts a general processing cluster 1000 in accordance with one embodiment.

[0016] FIG. 11 depicts a memory partition unit 1100 in accordance with one embodiment.

[0017] FIG. 12 depicts a streaming multiprocessor 1200 in accordance with one embodiment.

[0018] FIG. 13 depicts a processing system 1300 in accordance with one embodiment.

[0019] FIG. 14 depicts an exemplary processing system 1400 in accordance with another embodiment.

[0020] FIG. 15 depicts a graphics processing pipeline 1500 in accordance with one embodiment.

[0021] FIG. 16 depicts a data center in accordance with one embodiment.

[0022] FIG. 17 depicts a circuit system 1702 in accordance with one embodiment.

DETAILED DESCRIPTION

[0023] Mechanisms are described herein to exploit the inherent resiliency of deep learning inference workloads to improve the energy efficiency of computer processors (e.g., graphics processing unit) with these workloads.

[0024] Some embodiments of the disclosed mechanisms utilize a multiplier unit configured to multiply two low precision floating-point (FP) operands to generate an integer formatted (INT) result, and an adder configured to add the integer result to a third operand. Other embodiments utilize multiplier units configured to multiply two low-precision (e.g., FP4) VS-Quant floating-point operands to generate a first INT result, an adder to reduce the first INT results to a second INT result, and logic to multiply the second INT result by a low-precision floating-point scale factor. Still other embodiments utilize a multiplier unit configured to add two low-precision VS-Quant (e.g., LOG4) floating-point operands into a first sum, logic to convert a portion of the first sum into an INT result, and logic to convert the INT result to an INT product of the floating-point operands. Herein, “low precision floating-point” refers to floating-point data formats utilizing eight or fewer bits.

[0025] In various embodiments these mechanisms provide energy-accuracy tradeoffs in the computation of deep learning inference calculations. The mechanisms utilize energy-efficient floating point data path microarchitectures with integer accumulation, and enhanced mechanisms for per-vector scaled quantization (VS-Quant) of low-precision floating-point arguments. Herein, “FP” refers to floating point data formats, INT refers to integer data formats, and INT32 refers to the well-known 32-bit fixed point representation of integer data types. Exemplary embodiments are depicted utilizing INT32 accumulation, but the disclosed mechanisms are applicable to INT accumulation in other bit widths as well.

[0026] The FP8 floating point data formats utilized in the example embodiments herein comprise two encodings—E4M3 and E5M2, where the name encodes the number of exponent (E) and mantissa (M) bits. For this data format, the term “mantissa” refers to IEEE 754 standard’s trailing significant field (i.e. bits not including the implied leading 1 bit for normal floating point numbers). The E4M3 encoding may be particularly applicable for representing weight and activation tensors, and the E5M2 encoding may be particularly applicable for representing gradient tensors. Some networks may train with just the E4M3 or the E5M2 data types, whereas others may benefit from training with both types (or may maintain many fewer tensors in FP8).

[0027] FIG. 1 depicts a specification of the E4M3 and E5M2 encodings in S.E.M notation, where S is the sign bit, E is the exponent field (either 4 or 5 bits containing biased exponent), and M is either a 3- or a 2-bit mantissa. Values with a two (2) in the subscript are binary, otherwise they are decimal.

[0028] Quantization enables efficient acceleration of deep neural networks by reducing model memory footprint and exploiting low-cost integer math hardware units. Quantiza-

tion maps floating-point weights and activations in a trained model to low bit-width integer values using scale factors. Excessive quantization, reducing precision too aggressively, results in degradation of the accuracy of the model. When scale factors are shared at a coarse granularity across many dimensions of each tensor, effective precision of individual elements within the tensor are limited.

[0029] To reduce quantization-related accuracy loss, per-vector scale factors (VS-Quant) may be applied to subset (e.g., 16-64 element) vectors within a single dimension of a tensor. To achieve an efficient hardware implementation, the per-vector scale factors may be implemented with low bit width integers when calibrated using a two-level quantization scheme.

[0030] Exponent biasing may be utilized in the data encodings. Exponent bias controls the representable range of real values (e.g., by shifting the range along the real number line). Exponent biasing also enables the exponent to be represented as an unsigned value more amenable to comparison. A biased exponent may be converted into a signed range for addition or subtraction by subtracting the bias. One common calculation of an exponent bias is 2^{E-1} , where E is the number of exponent bits. The fields of the exponent value may be arranged such that the sign bit takes the most significant bit position, the biased exponent takes the middle position, then the significand occupies the least significant bits. This format enables high speed comparisons between floating-point numbers using fixed-point hardware. The E4M3 and E5M2 encodings may utilize exponent biases of 7 and 15, respectively.

[0031] The description of the following exemplary embodiments includes logic components such as XOR, addition, complementing, shifting, truncation, alignment, and so on. Unless otherwise indicated, these components may be implemented in any number of manners that are well-known and understood in the art.

[0032] FIG. 2 depicts a micro-architecture of a data path used in conventional vector multiply units **202** that operate on FP8 formatted operands. The data path comprises four stages. Stage 1, comprising a mantissa multiplier **204**, adder **206**, and XOR logic **208** in each multiply-accumulate unit, computes a product of the vectors by multiplying the mantissa, adding the exponents, and XORing the sign bits, respectively, of two operands A and B.

[0033] The product exponents from each multiply-accumulate unit are applied to a maximum selector **210** to select the maximum value thereof. The product exponent values are aligned for addition by shifting the product mantissas with a bitwise shifter **212**. These values are processed through a 2's complementer **214** and an adder **216** to perform vector addition and accumulation of the resulting values, along with the mantissa of a third operand C. The accumulated result is converted into an FP32 format via a normalizer **218**. FP32, also known as float32, refers to the well-known IEEE 32-bit, single-precision floating-point data format.

[0034] The maximum selector **210** and shifter **212** utilize the exponent sums output by the adders **206** to perform a data alignment function on the mantissa products output from the mantissa multipliers **204**, so that the mantissa products may be reduced by the adder **216**. Implementing this alignment function consumes area and power and increases cost/complexity. This alignment function may be

obviated utilizing pipeline designs in accordance with the embodiments described below.

[0035] Another component that increases area, power consumption, and cost/complexity is the normalizer **218**, which adjusts the exponent to conform to the standard IEEE FP32 format. Normalization in this manner may also be obviated in accordance with the embodiments below.

[0036] FIG. 3 depicts an embodiment micro-architecture for the data path of a vector multiply unit **302** (MAC). This micro-architecture may demonstrate improved area and energy efficiency over conventional mechanisms. Exponents of two operands A and B are added (adder **304**), mantissas are multiplied (mantissa multiplier **306**), and sign bits are XORed (XOR logic **308**). The depicted FP8 vector MAC implementation converts the product mantissa and product exponent to INT partial products using shift and rounding operations (rounder **310**, 2's complementer **312**, and shifter and rounder **314**). The exponent sum E_p determines the shift value that is applied by the shifter and rounder **314**. Due to the reduction in the number of exponent bits (to four bits) over conventional approaches, the utilization of alignment logic is obviated without necessity of an impractically wide adder **216** of the partial products from the multiply units **302**.

[0037] The exemplary data path in FIG. 3 utilizes FP8 formatted arguments with 4-bit exponents and 3-bit mantissas (herein, E4M3 format). This format provides a substantially reduced exponent range compared with high-precision floating point formats. The constrained exponent range enables direct conversion to INT data formats and obviates a need for the max computation and normalization operations of the data path depicted in FIG. 2. The data path may be readily adapted for use with E3M4 and E2M5 data formats as well.

[0038] Weight and activation operands may be scaled to fully utilize the exponent range during inference, which enables a reduction in the bit width of intermediate values (e.g., mantissa product (M_p) width and accumulation internal width (A_{IW})), thereby enabling the truncation of small values that have very little impact on the inference outcomes.

[0039] The direct conversion of partial products to integer format obviates exponent normalization and alignment before reduction. Truncating the mantissa product to M_p bits and truncating integer partial products to A_{IW} bits reduces the complexity of the shifter and rounder **314** logic, reduction adder **316**, and the number of flip-flops utilized in the pipeline. The adder **316** combines the partial product from the multiply unit **302** with a third operand (C) formatted as a 32-bit, 2s-complement integer. The third operand is for example a running sum obtained from prior multiply-accumulate iterations (e.g., the "D" value from a prior iteration).

[0040] The partial sum values may thus be accumulated and stored in INT32 format (unlike conventional approaches that accumulate/reduce in floating point). This obviates use of normalization and alignment operations and simplifies the rounding logic, which may now be carried out on INT32 formatted values instead of FP32 formatted values. These enhancements enable the FP8 data path to be implemented with two pipeline stages compared to the 4-stage pipeline of the traditional FP8 data path (FIG. 2). Although the accumulation/reduction could instead be performed in floating point (e.g., by including a normalizer **218** and/or alignment logic in the processing flow downstream of the adder **316**),

this may obviate or reduce some of the savings in area, power, and complexity of the implementation.

[0041] FIG. 4A depicts an embodiment to further lower precision in the data path of vector multiply units **402** utilizing VS-Quant FP4 data formats. The depicted embodiment comprises a data path for VS-Quant FP4 (e.g., E2M1 encoded) vectors and a per-vector FP4 (e.g., unsigned E3M1) scale factor. Utilization of 4-bit precision and quantized 4-bit precision enables a significant reduction in the area and energy consumption of the vector multiply-accumulate unit logic. Although the example embodiment utilizes the E2M1 (signed) and E3M1 (unsigned) formats specifically, other FP4 and quantized FP4 formats may also be utilized (e.g., E1M2, E0M3, E3M0 vectors with E4M0, E2M2, E1M3, E0M4 per-vector scale factors—see FIG. 4B) depending on the constraints of the implementation. In addition the bit width of per-vector scale factors may be increased beyond four bits (e.g., to eight bits) with some corresponding increase in logic size and/or complexity.

[0042] The VS-Quant FP4 data format comprises a vector of FP4 (E2M1, E1M2, E0M3) formatted elements and an FP4 or FP8 formatted scale factor applied to all elements in the vector. VS-Quant LOG4 comprises a vector of elements each represented in fixed-point exponent-only formats (E3.0, E2.1, E1.2) and an exponent bias scale factor applied to each element in the vector. Conventional block data formats such as Block FP and Microsoft-FP comprise a vector of fixed-point formatted elements and a scale factor that is applied only to the exponent of these elements.

[0043] In the data path, exponents of two operands A and B are added (adder **404**), mantissas are multiplied (mantissa multiplier **406**), and sign bits are XORed (XOR logic **408**). The product mantissa M_p is complemented based on the product sign bit S_p (2's complementer **410**). The result is shifted and truncated (shifter and truncater **412**) based on the product exponent E_p to generate a partial product.

[0044] The partial products from the multiply units **402** are reduced (added) in an adder **414**. Post-processing of the reduced result is then performed. The reduced result is multiplied with the product of M_{SA} and M_{SW} (multiplier **416** and multiplier **418**). M_{SA} represents a mantissa scale factor for input activations of the neural network layer being processed, and M_{SW} represents a mantissa scale factor for weights of the layer.

[0045] Because the mantissa scale factors are E3M1 format, the multiplier **416** comprises a two-bit multiplier yielding a four bit product. Multiplier **418** performs a four-bit multiply on the output of adder **414**.

[0046] The exponent scale factors (E_{SA} for activations and E_{SW} for weights) are added (adder **420**) and applied as a shift amount on the output of multiplier **418**, with appropriate truncation (shifter and truncater **422**). The result is added to the third 32-bit 2's-complement operand (C, e.g. a running sum) to produce the final output D (adder **424**).

[0047] The utilization of FP4 format, per-vector scaling introduces only a small overhead to scale the dot-product outputs and to perform accumulation in INT32 format.

[0048] FIG. 5 depicts an embodiment of a data path in which INT accumulation is utilized to generate partial dot-products that are then accumulated in an FP32 format. Similar to the embodiment depicted in FIG. 4A, the data path operates on VS-Quant FP4 (e.g., E2M1 encoded) vectors utilizing per-vector FP4 (e.g., unsigned E3M1) scale factors. The data path may utilize quantized FP4 data

formats such as E2M1, E3M1, E1M2, E0M3, and E3M0 vectors with E4M0, E2M2, E1M3, E0M4 per-vector scale factors. Some implementations may increase the bit width of the per-vector scale factors beyond four bits.

[0049] The adder **504** adds the exponents of two operands A and B, the mantissa multiplier **506** multiplies the mantissas, and the XOR logic **508** combines the sign bits. The 2's complementer **510** performs a two's complement on the product mantissa M_p based on the sign bit S_p . The result is shifted (shifter **512**) based on the product exponent E_p to generate a partial dot product.

[0050] The partial products from the multiply units **502** are reduced in the adder **514** and the reduced result is multiplied with the product of M_{SA} and M_{SW} (multiplier **516** and multiplier **518**), where M_{SA} represents the mantissa scale factor for input activations of the neural network layer being processed, and M_{SW} represents the mantissa scale factor for weights of the layer.

[0051] The adder **520** combines the exponent scale factors (E_{SA} for activations and E_{SW} for weights) and the output of the multiplier **518** is aligned (aligner **522**) by an amount determined by the maximum value of the combined exponent E_{sp} and the exponent of the partial accumulated value C. The final adder **524** computes the sum of the operands in FP32 format. The aligner **522** and adder **524** input results from the multipliers **518** and adders **520** along with the partial accumulated value C in FP32 format, and transform these inputs to the final product in FP32 format.

[0052] FIG. 6A depicts a data path of multiply-accumulate units **602** that operate on a data format referred to herein as VS-Quant LOG4. The base of the data format is determined as $\text{base}=2^{1/\text{base_factor}}$ and the exponent representation is Eq.r where q is the number of quotient bits and r is the number of remainder bits. The exponent may be represented as a fixed-point value (e.g. E3.0, E2.1, E1.2) format, for example. FIG. 6B depicts an E3.0/LOG4 data format. Utilizing VS-Quant LOG4 format, the value and precision of the operands is carried entirely in the exponent values (excepting the sign bit), so that pipeline processing of distinct mantissa values is obviated.

[0053] Each vector multiply-accumulate unit adds the exponents (adder **604**) of two operands A and B, including the exponent bias E_{bias} , converts the M-portion of the floating-point exponent sum to INT format using a look-up table **606**, and shifts the result by E bits of the exponent sum (shifter **608**). Use of a fast lookup table for the data type conversion is practical in both energy, area, and cost, due to the very low precision of the exponents.

[0054] Results after shifting are truncated (truncater **610**) and complemented (2's complementer **612**) based on the result of XORing the sign bits (ADD **614**). Reduction of the resulting partial products from the multiply-accumulate units, and addition of a third operand C, is performed in the adder tree **616** in integer format.

[0055] FIG. 7 depicts a neural network processor **700** embodied on a single chip. The neural network processor **700** may utilize a plurality of processing elements **702**, one or more of which may implement data path mechanisms in accordance with those described herein. The neural network processor **700** also comprises a global buffer **704** and controller **706**, which for example may be a RISC-V processor. The processing elements **702** and global buffer **704** communicate via the network-on-a-chip router or other interconnect technology (see the GPU implementations,

described further below). If a router is utilized, it may be implemented centrally or in distributed fashion as routers on each of the processing elements 702. The processing elements 702 utilize the router/interconnect to communicate with processing elements on the same package, or in some embodiments across packages via a network-on-a-package router 708.

[0056] FIG. 8 depicts, and a high level, an exemplary processing element 800 that may implement one or more of the processing elements 702. The processing element 800 includes a plurality of vector multiply-accumulate units 802, a weight buffer 604, an activation buffer 804, a router 806, a controller 808, an accumulation memory buffer 810, and a post-processor 612. The activation buffer 804 may, in one embodiment, be implemented as a dual-ported SRAM to receive activation values from the global buffer 704 or from other local or global processing elements, via the router 806 or other interconnect. The router 608 may be a component of a distributed network-on-a-chip router that in one embodiment comprises a serializer/de-serializer, packetizer, arbitrator, Advanced eXtensible Interface, and other components known in the art.

[0057] The weight buffer 812 may, in one embodiment, be implemented as a single-ported SRAM storing weight values. The weight values used by the vector multiply-accumulate units 802 may be “weight-stationary”, meaning they are not updated each clock cycle, but instead are updated only after the output activation values are computed for a particular layer of the deep neural network.

[0058] The accumulation memory buffer 810 may comprise one or more SRAM devices to store the output activations computed by the vector multiply-accumulate units 802. The router 806 communicates these output activations and control signals from the processing element 800 to other processing elements. “Output activation” refers to an activation output by a neuron in a neural network. An output activation is typically computed based on the input activations to the neuron and the weights applied to the input activations. “Input activation” refers to an activation received by a neuron in a neural network.

[0059] The processing element 800 may perform all operations of convolutional and fully-connected layers of a DNN efficiently, including multiply-accumulate, truncation, scaling, bias addition, ReLU, and pooling (these last five in the post-processor 814). The vector multiply-accumulate units 802 may operate on the same inputs using different filters. In one embodiment, each of the vector multiply-accumulate units 802 performs an eight-input-channel dot product and accumulates the result into the accumulation memory buffer 810 on each clock cycle. The weights stored in the weight buffer 812 are unchanged until the entire computation of output activations completes. Each processing element 800 reads the input activations in the activation buffer 804, performs the multiply-accumulate operations, and writes output activations to the accumulation memory buffer 810 on every clock cycle. The frequency at which the weight buffer 812 is accessed depends on the input activation matrix dimensions and the number of filters utilized.

[0060] The vector multiply-accumulate units 802 of each processing element 800 computes a portion of a wide dot-product-accumulate as a partial result and forwards the partial result to neighboring processing elements. “Dot-product-accumulate” refers to the computation of a dot product. A dot product is the sum of the products of the

corresponding entries of the two sequences (vectors) of numbers. Dot products are efficiently computed using vector multiply-accumulate units. “Multiply-accumulate unit” refers to a data processing circuit that carries out multiply-accumulate operations, which involve computing the product of two numbers and adding that product to an accumulator. Multiply-accumulate units may be referred to herein by their acronym, MAC or MAC unit. A multiply-accumulate unit carries out computations of the form $a \leftarrow a + (b * c)$. A vector multiply-accumulate unit computes the product of two vectors using an array of multipliers, then performs a reduction operation by adding all the outputs of multipliers to produce a partial sum, which is then added to an accumulator. The partial results are transformed into a final result by the post-processor 814 and communicated to the global buffer 704. The global buffer 704 acts as a staging area for the final multiply-accumulate results between layers of the deep neural network.

[0061] The accumulation memory buffer 810 receives outputs from the vector multiply-accumulate units 802. The central controller 706 distributes the weight values and activation values among the processing elements and utilizes the global memory buffer as a second-level buffer for the activation values. When processing images, the controller 706 configures processing by layers of the deep neural network spatially across the processing elements by input/output channel dimensions and temporally by image height/width.

[0062] The global buffer 704 stores both input activations and output activations from the processing elements 702 for distribution by the aforementioned transceivers to the processing elements via multicast. “Multicast” refers to a group communication mechanism whereby transmission of data is addressed to a group of destination devices (e.g., processing elements) simultaneously. Multicast can implement one-to-many or many-to-many distribution. Each of the processing elements 702 includes a router 806 to communicate, in one embodiment, 64 bits of data in, and 64 bits of data out, per clock cycle. This enables accumulation of partial sums for wide dot products that have their computation spatially tiled across the processing elements 702.

[0063] The data path mechanisms disclosed herein may be implemented in computing devices utilizing one or more graphic processing unit (GPU) and/or general purpose data processor (e.g., a central processing unit or CPU). Exemplary architectures will now be described that may be configured with mechanisms in accordance with those disclosed herein.

[0064] The following description may use certain acronyms and abbreviations as follows:

- [0065] “DPC” refers to a “data processing cluster”;
- [0066] “GPC” refers to a “general processing cluster”;
- [0067] “I/O” refers to a “input/output”;
- [0068] “L1 cache” refers to “level one cache”;
- [0069] “L2 cache” refers to “level two cache”;
- [0070] “LSU” refers to a “load/store unit”;
- [0071] “MMU” refers to a “memory management unit”;
- [0072] “MPC” refers to an “M-pipe controller”;
- [0073] “PPU” refers to a “parallel processing unit”;
- [0074] “PROP” refers to a “pre-raster operations unit”;
- [0075] “ROP” refers to a “raster operations”;
- [0076] “SFU” refers to a “special function unit”;
- [0077] “SM” refers to a “streaming multiprocessor”;

[0078] “Viewport SCC” refers to “viewport scale, cull, and clip”;

[0079] “WDX” refers to a “work distribution crossbar”;

[0080] “XBar” refers to a “crossbar”.

Parallel Processing Unit

[0081] FIG. 9 depicts a parallel processing unit 902, in accordance with an embodiment. In an embodiment, the parallel processing unit 902 is a multi-threaded processor that is implemented on one or more integrated circuit devices. The parallel processing unit 902 is a latency hiding architecture designed to process many threads in parallel. A thread (e.g., a thread of execution) is an instantiation of a set of instructions configured to be executed by the parallel processing unit 902. In an embodiment, the parallel processing unit 902 is a graphics processing unit (GPU) configured to implement a graphics rendering pipeline for processing three-dimensional (3D) graphics data in order to generate two-dimensional (2D) image data for display on a display device such as a liquid crystal display (LCD) device. In other embodiments, the parallel processing unit 902 may be utilized for performing general-purpose computations. While one exemplary parallel processor is provided herein for illustrative purposes, it should be strongly noted that such processor is set forth for illustrative purposes only, and that any processor may be employed to supplement and/or substitute for the same.

[0082] One or more parallel processing unit 902 modules may be configured to accelerate thousands of High Performance Computing (HPC), data center, and machine learning applications. The parallel processing unit 902 may be configured to accelerate numerous deep learning systems and applications including autonomous vehicle platforms, deep learning, high-accuracy speech, image, and text recognition systems, intelligent video analytics, molecular simulations, drug discovery, disease diagnosis, weather forecasting, big data analytics, astronomy, molecular dynamics simulation, financial modeling, robotics, factory automation, real-time language translation, online search optimizations, and personalized user recommendations, and the like.

[0083] As shown in FIG. 9, the parallel processing unit 902 includes an I/O unit 904, a front-end unit 906, a scheduler unit 908, a work distribution unit 910, a hub 912, a crossbar 914, one or more general processing cluster 1000 modules, and one or more memory partition unit 1100 modules. The parallel processing unit 902 may be connected to a host processor or other parallel processing unit 902 modules via one or more high-speed NVLink 916 interconnects. The parallel processing unit 902 may be connected to a host processor or other peripheral devices via an interconnect 918. The parallel processing unit 902 may also be connected to a local memory comprising a number of memory 920 devices. In an embodiment, the local memory may comprise a number of dynamic random access memory (DRAM) devices. The DRAM devices may be configured as a high-bandwidth memory (HBM) subsystem, with multiple DRAM dies stacked within each device. The memory 920 may comprise logic to configure the parallel processing unit 902 to carry out aspects of the techniques disclosed herein.

[0084] The NVLink 916 interconnect enables systems to scale and include one or more parallel processing unit 902 modules combined with one or more CPUs, supports cache coherence between the parallel processing unit 902 modules

and CPUs, and CPU mastering. Data and/or commands may be transmitted by the NVLink 916 through the hub 912 to/from other units of the parallel processing unit 902 such as one or more copy engines, a video encoder, a video decoder, a power management unit, etc. (not explicitly shown). The NVLink 916 is described in more detail in conjunction with FIG. 13.

[0085] The I/O unit 904 is configured to transmit and receive communications (e.g., commands, data, etc.) from a host processor (not shown) over the interconnect 918. The I/O unit 904 may communicate with the host processor directly via the interconnect 918 or through one or more intermediate devices such as a memory bridge. In an embodiment, the I/O unit 904 may communicate with one or more other processors, such as one or more parallel processing unit 902 modules via the interconnect 918. In an embodiment, the I/O unit 904 implements a Peripheral Component Interconnect Express (PCIe) interface for communications over a PCIe bus and the interconnect 918 is a PCIe bus. In alternative embodiments, the I/O unit 904 may implement other types of well-known interfaces for communicating with external devices.

[0086] The I/O unit 904 decodes packets received via the interconnect 918. In an embodiment, the packets represent commands configured to cause the parallel processing unit 902 to perform various operations. The I/O unit 904 transmits the decoded commands to various other units of the parallel processing unit 902 as the commands may specify. For example, some commands may be transmitted to the front-end unit 906. Other commands may be transmitted to the hub 912 or other units of the parallel processing unit 902 such as one or more copy engines, a video encoder, a video decoder, a power management unit, etc. (not explicitly shown). In other words, the I/O unit 904 is configured to route communications between and among the various logical units of the parallel processing unit 902.

[0087] In an embodiment, a program executed by the host processor encodes a command stream in a buffer that provides workloads to the parallel processing unit 902 for processing. A workload may comprise several instructions and data to be processed by those instructions. The buffer is a region in a memory that is accessible (e.g., read/write) by both the host processor and the parallel processing unit 902. For example, the I/O unit 904 may be configured to access the buffer in a system memory connected to the interconnect 918 via memory requests transmitted over the interconnect 918. In an embodiment, the host processor writes the command stream to the buffer and then transmits a pointer to the start of the command stream to the parallel processing unit 902. The front-end unit 906 receives pointers to one or more command streams. The front-end unit 906 manages the one or more streams, reading commands from the streams and forwarding commands to the various units of the parallel processing unit 902.

[0088] The front-end unit 906 is coupled to a scheduler unit 908 that configures the various general processing cluster 1000 modules to process tasks defined by the one or more streams. The scheduler unit 908 is configured to track state information related to the various tasks managed by the scheduler unit 908. The state may indicate which general processing cluster 1000 a task is assigned to, whether the task is active or inactive, a priority level associated with the task, and so forth. The scheduler unit 908 manages the

execution of a plurality of tasks on the one or more general processing cluster **1000** modules.

[0089] The scheduler unit **908** is coupled to a work distribution unit **910** that is configured to dispatch tasks for execution on the general processing cluster **1000** modules. The work distribution unit **910** may track a number of scheduled tasks received from the scheduler unit **908**. In an embodiment, the work distribution unit **910** manages a pending task pool and an active task pool for each of the general processing cluster **1000** modules. The pending task pool may comprise a number of slots (e.g., 32 slots) that contain tasks assigned to be processed by a particular general processing cluster **1000**. The active task pool may comprise a number of slots (e.g., 4 slots) for tasks that are actively being processed by the general processing cluster **1000** modules. As a general processing cluster **1000** finishes the execution of a task, that task is evicted from the active task pool for the general processing cluster **1000** and one of the other tasks from the pending task pool is selected and scheduled for execution on the general processing cluster **1000**. If an active task has been idle on the general processing cluster **1000**, such as while waiting for a data dependency to be resolved, then the active task may be evicted from the general processing cluster **1000** and returned to the pending task pool while another task in the pending task pool is selected and scheduled for execution on the general processing cluster **1000**.

[0090] The work distribution unit **910** communicates with the one or more general processing cluster **1000** modules via crossbar **914**. The crossbar **914** is an interconnect network that couples many of the units of the parallel processing unit **902** to other units of the parallel processing unit **902**. For example, the crossbar **914** may be configured to couple the work distribution unit **910** to a particular general processing cluster **1000**. Although not shown explicitly, one or more other units of the parallel processing unit **902** may also be connected to the crossbar **914** via the hub **912**.

[0091] The tasks are managed by the scheduler unit **908** and dispatched to a general processing cluster **1000** by the work distribution unit **910**. The general processing cluster **1000** is configured to process the task and generate results. The results may be consumed by other tasks within the general processing cluster **1000**, routed to a different general processing cluster **1000** via the crossbar **914**, or stored in the memory **920**. The results can be written to the memory **920** via the memory partition unit **1100** modules, which implement a memory interface for reading and writing data to/from the memory **920**. The results can be transmitted to another parallel processing unit **902** or CPU via the NVLink **916**. In an embodiment, the parallel processing unit **902** includes a number U of memory partition unit **1100** modules that is equal to the number of separate and distinct memory **920** devices coupled to the parallel processing unit **902**. A memory partition unit **1100** will be described in more detail below in conjunction with FIG. 11.

[0092] In an embodiment, a host processor executes a driver kernel that implements an application programming interface (API) that enables one or more applications executing on the host processor to schedule operations for execution on the parallel processing unit **902**. In an embodiment, multiple compute applications are simultaneously executed by the parallel processing unit **902** and the parallel processing unit **902** provides isolation, quality of service (QoS), and independent address spaces for the multiple compute appli-

cations. An application may generate instructions (e.g., API calls) that cause the driver kernel to generate one or more tasks for execution by the parallel processing unit **902**. The driver kernel outputs tasks to one or more streams being processed by the parallel processing unit **902**. Each task may comprise one or more groups of related threads, referred to herein as a warp. In an embodiment, a warp comprises 32 related threads that may be executed in parallel. Cooperating threads may refer to a plurality of threads including instructions to perform the task and that may exchange data through shared memory. Threads and cooperating threads are described in more detail in conjunction with FIG. 12.

[0093] FIG. 10 depicts a general processing cluster **1000** of the parallel processing unit **902** of FIG. 9, in accordance with an embodiment. As shown in FIG. 10, each general processing cluster **1000** includes a number of hardware units for processing tasks. In an embodiment, each general processing cluster **1000** includes a pipeline manager **1002**, a pre-raster operations unit **1004**, a raster engine **1006**, a work distribution crossbar **1008**, a memory management unit **1010**, and one or more data processing cluster **1012**. It will be appreciated that the general processing cluster **1000** of FIG. 10 may include other hardware units in lieu of or in addition to the units shown in FIG. 10.

[0094] In an embodiment, the operation of the general processing cluster **1000** is controlled by the pipeline manager **1002**. The pipeline manager **1002** manages the configuration of the one or more data processing cluster **1012** modules for processing tasks allocated to the general processing cluster **1000**. In an embodiment, the pipeline manager **1002** may configure at least one of the one or more data processing cluster **1012** modules to implement at least a portion of a graphics rendering pipeline. For example, a data processing cluster **1012** may be configured to execute a vertex shader program on the programmable streaming multiprocessor **1200**. The pipeline manager **1002** may also be configured to route packets received from the work distribution unit **910** to the appropriate logical units within the general processing cluster **1000**. For example, some packets may be routed to fixed function hardware units in the pre-raster operations unit **1004** and/or raster engine **1006** while other packets may be routed to the data processing cluster **1012** modules for processing by the primitive engine **1014** or the streaming multiprocessor **1200**. In an embodiment, the pipeline manager **1002** may configure at least one of the one or more data processing cluster **1012** modules to implement a neural network model and/or a computing pipeline.

[0095] The pre-raster operations unit **1004** is configured to route data generated by the raster engine **1006** and the data processing cluster **1012** modules to a Raster Operations (ROP) unit, described in more detail in conjunction with FIG. 11. The pre-raster operations unit **1004** may also be configured to perform optimizations for color blending, organize pixel data, perform address translations, and the like.

[0096] The raster engine **1006** includes a number of fixed function hardware units configured to perform various raster operations. In an embodiment, the raster engine **1006** includes a setup engine, a coarse raster engine, a culling engine, a clipping engine, a fine raster engine, and a tile coalescing engine. The setup engine receives transformed vertices and generates plane equations associated with the geometric primitive defined by the vertices. The plane

equations are transmitted to the coarse raster engine to generate coverage information (e.g., an x, y coverage mask for a tile) for the primitive. The output of the coarse raster engine is transmitted to the culling engine where fragments associated with the primitive that fail a z-test are culled, and transmitted to a clipping engine where fragments lying outside a viewing frustum are clipped. Those fragments that survive clipping and culling may be passed to the fine raster engine to generate attributes for the pixel fragments based on the plane equations generated by the setup engine. The output of the raster engine **1006** comprises fragments to be processed, for example, by a fragment shader implemented within a data processing cluster **1012**.

[0097] Each data processing cluster **1012** included in the general processing cluster **1000** includes an M-pipe controller **1016**, a primitive engine **1014**, and one or more streaming multiprocessor **1200** modules. The M-pipe controller **1016** controls the operation of the data processing cluster **1012**, routing packets received from the pipeline manager **1002** to the appropriate units in the data processing cluster **1012**. For example, packets associated with a vertex may be routed to the primitive engine **1014**, which is configured to fetch vertex attributes associated with the vertex from the memory **920**. In contrast, packets associated with a shader program may be transmitted to the streaming multiprocessor **1200**.

[0098] The streaming multiprocessor **1200** comprises a programmable streaming processor that is configured to process tasks represented by a number of threads. Each streaming multiprocessor **1200** is multi-threaded and configured to execute a plurality of threads (e.g., 32 threads) from a particular group of threads concurrently. In an embodiment, the streaming multiprocessor **1200** implements a Single-Instruction, Multiple-Data (SIMD) architecture where each thread in a group of threads (e.g., a warp) is configured to process a different set of data based on the same set of instructions. All threads in the group of threads execute the same instructions. In another embodiment, the streaming multiprocessor **1200** implements a Single-Instruction, Multiple Thread (SIMT) architecture where each thread in a group of threads is configured to process a different set of data based on the same set of instructions, but where individual threads in the group of threads are allowed to diverge during execution. In an embodiment, a program counter, call stack, and execution state is maintained for each warp, enabling concurrency between warps and serial execution within warps when threads within the warp diverge. In another embodiment, a program counter, call stack, and execution state is maintained for each individual thread, enabling equal concurrency between all threads, within and between warps. When execution state is maintained for each individual thread, threads executing the same instructions may be converged and executed in parallel for maximum efficiency. The streaming multiprocessor **1200** will be described in more detail below in conjunction with FIG. 12.

[0099] The memory management unit **1010** provides an interface between the general processing cluster **1000** and the memory partition unit **1100**. The memory management unit **1010** may provide translation of virtual addresses into physical addresses, memory protection, and arbitration of memory requests. In an embodiment, the memory management unit **1010** provides one or more translation lookaside

buffers (TLBs) for performing translation of virtual addresses into physical addresses in the memory **920**.

[0100] FIG. 11 depicts a memory partition unit **1100** of the parallel processing unit **902** of FIG. 9, in accordance with an embodiment. As shown in FIG. 11, the memory partition unit **1100** includes a raster operations unit **1102**, a level two cache **1104**, and a memory interface **1106**. The memory interface **1106** is coupled to the memory **920**. Memory interface **1106** may implement 32, 64, 128, 1024-bit data buses, or the like, for high-speed data transfer. In an embodiment, the parallel processing unit **902** incorporates U memory interface **1106** modules, one memory interface **1106** per pair of memory partition unit **1100** modules, where each pair of memory partition unit **1100** modules is connected to a corresponding memory **920** device. For example, parallel processing unit **902** may be connected to up to Y memory **920** devices, such as high bandwidth memory stacks or graphics double-data-rate, version 5, synchronous dynamic random access memory, or other types of persistent storage.

[0101] In an embodiment, the memory interface **1106** implements an HBM2 memory interface and Y equals half U. In an embodiment, the HBM2 memory stacks are located on the same physical package as the parallel processing unit **902**, providing substantial power and area savings compared with conventional GDDR5 SDRAM systems. In an embodiment, each HBM2 stack includes four memory dies and Y equals 4, with HBM2 stack including two 128-bit channels per die for a total of 8 channels and a data bus width of 1024 bits.

[0102] In an embodiment, the memory **920** supports Single-Error Correcting Double-Error Detecting (SECDED) Error Correction Code (ECC) to protect data. ECC provides higher reliability for compute applications that are sensitive to data corruption. Reliability is especially important in large-scale cluster computing environments where parallel processing unit **902** modules process very large datasets and/or run applications for extended periods.

[0103] In an embodiment, the parallel processing unit **902** implements a multi-level memory hierarchy. In an embodiment, the memory partition unit **1100** supports a unified memory to provide a single unified virtual address space for CPU and parallel processing unit **902** memory, enabling data sharing between virtual memory systems. In an embodiment the frequency of accesses by a parallel processing unit **902** to memory located on other processors is traced to ensure that memory pages are moved to the physical memory of the parallel processing unit **902** that is accessing the pages more frequently. In an embodiment, the NVLink **916** supports address translation services allowing the parallel processing unit **902** to directly access a CPU's page tables and providing full access to CPU memory by the parallel processing unit **902**.

[0104] In an embodiment, copy engines transfer data between multiple parallel processing unit **902** modules or between parallel processing unit **902** modules and CPUs. The copy engines can generate page faults for addresses that are not mapped into the page tables. The memory partition unit **1100** can then service the page faults, mapping the addresses into the page table, after which the copy engine can perform the transfer. In a conventional system, memory is pinned (e.g., non-pageable) for multiple copy engine operations between multiple processors, substantially reducing the available memory. With hardware page faulting,

addresses can be passed to the copy engines without worrying if the memory pages are resident, and the copy process is transparent.

[0105] Data from the memory 920 or other system memory may be fetched by the memory partition unit 1100 and stored in the level two cache 1104, which is located on-chip and is shared between the various general processing cluster 1000 modules. As shown, each memory partition unit 1100 includes a portion of the level two cache 1104 associated with a corresponding memory 920 device. Lower level caches may then be implemented in various units within the general processing cluster 1000 modules. For example, each of the streaming multiprocessor 1200 modules may implement an L1 cache. The L1 cache is private memory that is dedicated to a particular streaming multiprocessor 1200. Data from the level two cache 1104 may be fetched and stored in each of the L1 caches for processing in the functional units of the streaming multiprocessor 1200 modules. The level two cache 1104 is coupled to the memory interface 1106 and the crossbar 914.

[0106] The raster operations unit 1102 performs graphics raster operations related to pixel color, such as color compression, pixel blending, and the like. The raster operations unit 1102 also implements depth testing in conjunction with the raster engine 1006, receiving a depth for a sample location associated with a pixel fragment from the culling engine of the raster engine 1006. The depth is tested against a corresponding depth in a depth buffer for a sample location associated with the fragment. If the fragment passes the depth test for the sample location, then the raster operations unit 1102 updates the depth buffer and transmits a result of the depth test to the raster engine 1006. It will be appreciated that the number of partition memory partition unit 1100 modules may be different than the number of general processing cluster 1000 modules and, therefore, each raster operations unit 1102 may be coupled to each of the general processing cluster 1000 modules. The raster operations unit 1102 tracks packets received from the different general processing cluster 1000 modules and determines which general processing cluster 1000 that a result generated by the raster operations unit 1102 is routed to through the crossbar 914. Although the raster operations unit 1102 is included within the memory partition unit 1100 in FIG. 11, in other embodiment, the raster operations unit 1102 may be outside of the memory partition unit 1100. For example, the raster operations unit 1102 may reside in the general processing cluster 1000 or another unit.

[0107] FIG. 12 illustrates the streaming multiprocessor 1200 of FIG. 10, in accordance with an embodiment. As shown in FIG. 12, the streaming multiprocessor 1200 includes an instruction cache 1202, one or more scheduler unit 1204 modules (e.g., such as scheduler unit 908), a register file 1206, one or more processing core 1208 modules, one or more special function unit 1210 modules, one or more load/store unit 1212 modules, an interconnect network 1214, and a shared memory/L1 cache 1216.

[0108] As described above, the work distribution unit 910 dispatches tasks for execution on the general processing cluster 1000 modules of the parallel processing unit 902. The tasks are allocated to a particular data processing cluster 1012 within a general processing cluster 1000 and, if the task is associated with a shader program, the task may be allocated to a streaming multiprocessor 1200. The scheduler unit 908 receives the tasks from the work distribution unit

910 and manages instruction scheduling for one or more thread blocks assigned to the streaming multiprocessor 1200. The scheduler unit 1204 schedules thread blocks for execution as warps of parallel threads, where each thread block is allocated at least one warp. In an embodiment, each warp executes 32 threads. The scheduler unit 1204 may manage a plurality of different thread blocks, allocating the warps to the different thread blocks and then dispatching instructions from the plurality of different cooperative groups to the various functional units (e.g., core 1208 modules, special function unit 1210 modules, and load/store unit 1212 modules) during each clock cycle.

[0109] Cooperative Groups is a programming model for organizing groups of communicating threads that allows developers to express the granularity at which threads are communicating, enabling the expression of richer, more efficient parallel decompositions. Cooperative launch APIs support synchronization amongst thread blocks for the execution of parallel algorithms. Conventional programming models provide a single, simple construct for synchronizing cooperating threads: a barrier across all threads of a thread block (e.g., the `syncthreads()` function). However, programmers would often like to define groups of threads at smaller than thread block granularities and synchronize within the defined groups to enable greater performance, design flexibility, and software reuse in the form of collective group-wide function interfaces.

[0110] Cooperative Groups enables programmers to define groups of threads explicitly at sub-block (e.g., as small as a single thread) and multi-block granularities, and to perform collective operations such as synchronization on the threads in a cooperative group. The programming model supports clean composition across software boundaries, so that libraries and utility functions can synchronize safely within their local context without having to make assumptions about convergence. Cooperative Groups primitives enable new patterns of cooperative parallelism, including producer-consumer parallelism, opportunistic parallelism, and global synchronization across an entire grid of thread blocks.

[0111] A dispatch 1218 unit is configured within the scheduler unit 1204 to transmit instructions to one or more of the functional units. In one embodiment, the scheduler unit 1204 includes two dispatch 1218 units that enable two different instructions from the same warp to be dispatched during each clock cycle. In alternative embodiments, each scheduler unit 1204 may include a single dispatch 1218 unit or additional dispatch 1218 units.

[0112] Each streaming multiprocessor 1200 includes a register file 1206 that provides a set of registers for the functional units of the streaming multiprocessor 1200. In an embodiment, the register file 1206 is divided between each of the functional units such that each functional unit is allocated a dedicated portion of the register file 1206. In another embodiment, the register file 1206 is divided between the different warps being executed by the streaming multiprocessor 1200. The register file 1206 provides temporary storage for operands connected to the data paths of the functional units.

[0113] Each streaming multiprocessor 1200 comprises L processing core 1208 modules. In an embodiment, the streaming multiprocessor 1200 includes a large number (e.g., 128, etc.) of distinct processing core 1208 modules. Each core 1208 may include a fully-pipelined, single-precision, double-precision, and/or mixed precision processing

unit that includes a floating point arithmetic logic unit and an integer arithmetic logic unit. In an embodiment, the floating point arithmetic logic units implement the IEEE 754-2008 standard for floating point arithmetic. In an embodiment, the core **1208** modules include 64 single-precision (32-bit) floating point cores, 64 integer cores, 32 double-precision (64-bit) floating point cores, and 8 tensor cores.

[0114] Tensor cores configured to perform matrix operations, and, in an embodiment, one or more tensor cores are included in the core **1208** modules. In particular, the tensor cores are configured to perform deep learning matrix arithmetic, such as convolution operations for neural network training and inferencing. In an embodiment, each tensor core operates on a 4x4 matrix and performs a matrix multiply and accumulate operation $D=A'B+C$, where A, B, C, and D are 4x4 matrices.

[0115] In an embodiment, the matrix multiply inputs A and B are 16-bit floating point matrices, while the accumulation matrices C and D may be 16-bit floating point or 32-bit floating point matrices. Tensor Cores operate on 16-bit floating point input data with 32-bit floating point accumulation. The 16-bit floating point multiply requires 64 operations and results in a full precision product that is then accumulated using 32-bit floating point addition with the other intermediate products for a 4x4x4 matrix multiply. In practice, Tensor Cores are used to perform much larger two-dimensional or higher dimensional matrix operations, built up from these smaller elements. An API, such as CUDA 9 C++ API, exposes specialized matrix load, matrix multiply and accumulate, and matrix store operations to efficiently use Tensor Cores from a CUDA-C++ program. At the CUDA level, the warp-level interface assumes 16x16 size matrices spanning all 32 threads of the warp.

[0116] Each streaming multiprocessor **1200** also comprises M special function unit **1210** modules that perform special functions (e.g., attribute evaluation, reciprocal square root, and the like). In an embodiment, the special function unit **1210** modules may include a tree traversal unit configured to traverse a hierarchical tree data structure. In an embodiment, the special function unit **1210** modules may include texture unit configured to perform texture map filtering operations. In an embodiment, the texture units are configured to load texture maps (e.g., a 2D array of texels) from the memory **920** and sample the texture maps to produce sampled texture values for use in shader programs executed by the streaming multiprocessor **1200**. In an embodiment, the texture maps are stored in the shared memory/L1 cache **1216**. The texture units implement texture operations such as filtering operations using mip-maps (e.g., texture maps of varying levels of detail). In an embodiment, each streaming multiprocessor **1200** includes two texture units.

[0117] Each streaming multiprocessor **1200** also comprises N load/store unit **1212** modules that implement load and store operations between the shared memory/L1 cache **1216** and the register file **1206**. Each streaming multiprocessor **1200** includes an interconnect network **1214** that connects each of the functional units to the register file **1206** and the load/store unit **1212** to the register file **1206** and shared memory/L1 cache **1216**. In an embodiment, the interconnect network **1214** is a crossbar that can be configured to connect any of the functional units to any of the registers in the register file **1206** and connect the load/store

unit **1212** modules to the register file **1206** and memory locations in shared memory/L1 cache **1216**.

[0118] The shared memory/L1 cache **1216** is an array of on-chip memory that allows for data storage and communication between the streaming multiprocessor **1200** and the primitive engine **1014** and between threads in the streaming multiprocessor **1200**. In an embodiment, the shared memory/L1 cache **1216** comprises 128 KB of storage capacity and is in the path from the streaming multiprocessor **1200** to the memory partition unit **1100**. The shared memory/L1 cache **1216** can be used to cache reads and writes. One or more of the shared memory/L1 cache **1216**, level two cache **1104**, and memory **920** are backing stores.

[0119] Combining data cache and shared memory functionality into a single memory block provides the best overall performance for both types of memory accesses. The capacity is usable as a cache by programs that do not use shared memory. For example, if shared memory is configured to use half of the capacity, texture and load/store operations can use the remaining capacity. Integration within the shared memory/L1 cache **1216** enables the shared memory/L1 cache **1216** to function as a high-throughput conduit for streaming data while simultaneously providing high-bandwidth and low-latency access to frequently reused data.

[0120] When configured for general purpose parallel computation, a simpler configuration can be used compared with graphics processing. Specifically, the fixed function graphics processing units shown in FIG. 9, are bypassed, creating a much simpler programming model. In the general purpose parallel computation configuration, the work distribution unit **910** assigns and distributes blocks of threads directly to the data processing cluster **1012** modules. The threads in a block execute the same program, using a unique thread ID in the calculation to ensure each thread generates unique results, using the streaming multiprocessor **1200** to execute the program and perform calculations, shared memory/L1 cache **1216** to communicate between threads, and the load/store unit **1212** to read and write global memory through the shared memory/L1 cache **1216** and the memory partition unit **1100**. When configured for general purpose parallel computation, the streaming multiprocessor **1200** can also write commands that the scheduler unit **908** can use to launch new work on the data processing cluster **1012** modules.

[0121] The parallel processing unit **902** may be included in a desktop computer, a laptop computer, a tablet computer, servers, supercomputers, a smart-phone (e.g., a wireless, hand-held device), personal digital assistant (PDA), a digital camera, a vehicle, a head mounted display, a hand-held electronic device, and the like. In an embodiment, the parallel processing unit **902** is embodied on a single semiconductor substrate. In another embodiment, the parallel processing unit **902** is included in a system-on-a-chip (SoC) along with one or more other devices such as additional parallel processing unit **902** modules, the memory **920**, a reduced instruction set computer (RISC) CPU, a memory management unit (MMU), a digital-to-analog converter (DAC), and the like.

[0122] In an embodiment, the parallel processing unit **902** may be included on a graphics card that includes one or more memory devices. The graphics card may be configured to interface with a PCIe slot on a motherboard of a desktop computer. In yet another embodiment, the parallel process-

ing unit **902** may be an integrated graphics processing unit (iGPU) or parallel processor included in the chipset of the motherboard.

Exemplary Computing System

[0123] Systems with multiple GPUs and CPUs are used in a variety of industries as developers expose and leverage more parallelism in applications such as artificial intelligence computing. High-performance GPU-accelerated systems with tens to many thousands of compute nodes are deployed in data centers, research facilities, and supercomputers to solve ever larger problems. As the number of processing devices within the high-performance systems increases, the communication and data transfer mechanisms need to scale to support the increased bandwidth.

[0124] FIG. 13 is a conceptual diagram of a processing system **1300** implemented using the parallel processing unit **902** of FIG. 9, in accordance with an embodiment. The processing system **1300** includes a central processing unit **1302**, switch **1304**, and multiple parallel processing unit **902** modules each and respective memory **920** modules. The NVLink **916** provides high-speed communication links between each of the parallel processing unit **902** modules. Although a particular number of NVLink **916** and interconnect **918** connections are illustrated in FIG. 13, the number of connections to each parallel processing unit **902** and the central processing unit **1302** may vary. The switch **1304** interfaces between the interconnect **918** and the central processing unit **1302**. The parallel processing unit **902** modules, memory **920** modules, and NVLink **916** connections may be situated on a single semiconductor platform to form a parallel processing module **1306**. In an embodiment, the switch **1304** supports two or more protocols to interface between various different connections and/or links.

[0125] In another embodiment (not shown), the NVLink **916** provides one or more high-speed communication links between each of the parallel processing unit modules (parallel processing unit **902**, parallel processing unit **902**, parallel processing unit **902**, and parallel processing unit **902**) and the central processing unit **1302** and the switch **1304** interfaces between the interconnect **918** and each of the parallel processing unit modules. The parallel processing unit modules, memory **920** modules, and interconnect **918** may be situated on a single semiconductor platform to form a parallel processing module **1306**. In yet another embodiment (not shown), the interconnect **918** provides one or more communication links between each of the parallel processing unit modules and the central processing unit **1302** and the switch **1304** interfaces between each of the parallel processing unit modules using the NVLink **916** to provide one or more high-speed communication links between the parallel processing unit modules. In another embodiment (not shown), the NVLink **916** provides one or more high-speed communication links between the parallel processing unit modules and the central processing unit **1302** through the switch **1304**. In yet another embodiment (not shown), the interconnect **918** provides one or more communication links between each of the parallel processing unit modules directly. One or more of the NVLink **916** high-speed communication links may be implemented as a physical NVLink interconnect or either an on-chip or on-die interconnect using the same protocol as the NVLink **916**.

[0126] In the context of the present description, a single semiconductor platform may refer to a sole unitary semi-

conductor-based integrated circuit fabricated on a die or chip. It should be noted that the term single semiconductor platform may also refer to multi-chip modules with increased connectivity which simulate on-chip operation and make substantial improvements over utilizing a conventional bus implementation. Of course, the various circuits or devices may also be situated separately or in various combinations of semiconductor platforms per the desires of the user. Alternately, the parallel processing module **1306** may be implemented as a circuit board substrate and each of the parallel processing unit modules and/or memory **920** modules may be packaged devices. In an embodiment, the central processing unit **1302**, switch **1304**, and the parallel processing module **1306** are situated on a single semiconductor platform.

[0127] In an embodiment, the signaling rate of each NVLink **916** is 20 to 25 Gigabits/second and each parallel processing unit module includes six NVLink **916** interfaces (as shown in FIG. 13, five NVLink **916** interfaces are included for each parallel processing unit module). Each NVLink **916** provides a data transfer rate of 25 Gigabytes/second in each direction, with six links providing 300 Gigabytes/second. The NVLink **916** can be used exclusively for PPU-to-PPU communication as shown in FIG. 13, or some combination of PPU-to-PPU and PPU-to-CPU, when the central processing unit **1302** also includes one or more NVLink **916** interfaces.

[0128] In an embodiment, the NVLink **916** allows direct load/store/atomic access from the central processing unit **1302** to each parallel processing unit module's memory **920**. In an embodiment, the NVLink **916** supports coherency operations, allowing data read from the memory **920** modules to be stored in the cache hierarchy of the central processing unit **1302**, reducing cache access latency for the central processing unit **1302**. In an embodiment, the NVLink **916** includes support for Address Translation Services (ATS), enabling the parallel processing unit module to directly access page tables within the central processing unit **1302**. One or more of the NVLink **916** may also be configured to operate in a low-power mode.

[0129] FIG. 14 depicts an exemplary processing system **1400** in which the various architecture and/or functionality of the various previous embodiments may be implemented. As shown, an exemplary processing system **1400** is provided including at least one central processing unit **1302** that is connected to a communications bus **1402**. The communication communications bus **1402** may be implemented using any suitable protocol, such as PCI (Peripheral Component Interconnect), PCI-Express, AGP (Accelerated Graphics Port), HyperTransport, or any other bus or point-to-point communication protocol(s). The exemplary processing system **1400** also includes a main memory **1404**. Control logic (software) and data are stored in the main memory **1404** which may take the form of random access memory (RAM).

[0130] The exemplary processing system **1400** also includes input devices **1406**, the parallel processing module **1306**, and display devices **1408**, e.g. a conventional CRT (cathode ray tube), LCD (liquid crystal display), LED (light emitting diode), plasma display or the like. User input may be received from the input devices **1406**, e.g., keyboard, mouse, touchpad, microphone, and the like. Each of the foregoing modules and/or devices may even be situated on a single semiconductor platform to form the exemplary

processing system **1400**. Alternately, the various modules may also be situated separately or in various combinations of semiconductor platforms per the desires of the user.

[0131] Further, the exemplary processing system **1400** may be coupled to a network (e.g., a telecommunications network, local area network (LAN), wireless network, wide area network (WAN) such as the Internet, peer-to-peer network, cable network, or the like) through a network interface **1410** for communication purposes.

[0132] The exemplary processing system **1400** may also include a secondary storage (not shown). The secondary storage includes, for example, a hard disk drive and/or a removable storage drive, representing a floppy disk drive, a magnetic tape drive, a compact disk drive, digital versatile disk (DVD) drive, recording device, universal serial bus (USB) flash memory. The removable storage drive reads from and/or writes to a removable storage unit in a well-known manner.

[0133] Computer programs, or computer control logic algorithms, may be stored in the main memory **1404** and/or the secondary storage. Such computer programs, when executed, enable the exemplary processing system **1400** to perform various functions. The main memory **1404**, the storage, and/or any other storage are possible examples of computer-readable media.

[0134] The architecture and/or functionality of the various previous figures may be implemented in the context of a general computer system, a circuit board system, a game console system dedicated for entertainment purposes, an application-specific system, and/or any other desired system. For example, the exemplary processing system **1400** may take the form of a desktop computer, a laptop computer, a tablet computer, servers, supercomputers, a smart-phone (e.g., a wireless, hand-held device), personal digital assistant (PDA), a digital camera, a vehicle, a head mounted display, a hand-held electronic device, a mobile phone device, a television, workstation, game consoles, embedded system, and/or any other type of logic.

[0135] While various embodiments have been described above, it should be understood that they have been presented by way of example only, and not limitation. Thus, the breadth and scope of a preferred embodiment should not be limited by any of the above-described exemplary embodiments, but should be defined only in accordance with the following claims and their equivalents.

Graphics Processing Pipeline

[0136] FIG. **15** is a conceptual diagram of a graphics processing pipeline **1500** implemented by the parallel processing unit **902** of FIG. **9**, in accordance with an embodiment. In an embodiment, the parallel processing unit **902** comprises a graphics processing unit (GPU). The parallel processing unit **902** is configured to receive commands that specify shader programs for processing graphics data. Graphics data may be defined as a set of primitives such as points, lines, triangles, quads, triangle strips, and the like. Typically, a primitive includes data that specifies a number of vertices for the primitive (e.g., in a model-space coordinate system) as well as attributes associated with each vertex of the primitive. The parallel processing unit **902** can be configured to process the graphics primitives to generate a frame buffer (e.g., pixel data for each of the pixels of the display).

[0137] An application writes model data for a scene (e.g., a collection of vertices and attributes) to a memory such as a system memory or memory **920**. The model data defines each of the objects that may be visible on a display. The application then makes an API call to the driver kernel that requests the model data to be rendered and displayed. The driver kernel reads the model data and writes commands to the one or more streams to perform operations to process the model data. The commands may reference different shader programs to be implemented on the streaming multiprocessor **1200** modules of the parallel processing unit **902** including one or more of a vertex shader, hull shader, domain shader, geometry shader, and a pixel shader. For example, one or more of the streaming multiprocessor **1200** modules may be configured to execute a vertex shader program that processes a number of vertices defined by the model data. In an embodiment, the different streaming multiprocessor **1200** modules may be configured to execute different shader programs concurrently. For example, a first subset of streaming multiprocessor **1200** modules may be configured to execute a vertex shader program while a second subset of streaming multiprocessor **1200** modules may be configured to execute a pixel shader program. The first subset of streaming multiprocessor **1200** modules processes vertex data to produce processed vertex data and writes the processed vertex data to the level two cache **1104** and/or the memory **920**. After the processed vertex data is rasterized (e.g., transformed from three-dimensional data into two-dimensional data in screen space) to produce fragment data, the second subset of streaming multiprocessor **1200** modules executes a pixel shader to produce processed fragment data, which is then blended with other processed fragment data and written to the frame buffer in memory **920**. The vertex shader program and pixel shader program may execute concurrently, processing different data from the same scene in a pipelined fashion until all of the model data for the scene has been rendered to the frame buffer. Then, the contents of the frame buffer are transmitted to a display controller for display on a display device.

[0138] The graphics processing pipeline **1500** is an abstract flow diagram of the processing steps implemented to generate 2D computer-generated images from 3D geometry data. As is well-known, pipeline architectures may perform long latency operations more efficiently by splitting up the operation into a plurality of stages, where the output of each stage is coupled to the input of the next successive stage. Thus, the graphics processing pipeline **1500** receives input data **601** that is transmitted from one stage to the next stage of the graphics processing pipeline **1500** to generate output data **1502**. In an embodiment, the graphics processing pipeline **1500** may represent a graphics processing pipeline defined by the OpenGL® API. As an option, the graphics processing pipeline **1500** may be implemented in the context of the functionality and architecture of the previous Figures and/or any subsequent Figure(s).

[0139] As shown in FIG. **15**, the graphics processing pipeline **1500** comprises a pipeline architecture that includes a number of stages. The stages include, but are not limited to, a data assembly **1504** stage, a vertex shading **1506** stage, a primitive assembly **1508** stage, a geometry shading **1510** stage, a viewport SCC **1512** stage, a rasterization **1514** stage, a fragment shading **1516** stage, and a raster operations **1518** stage. In an embodiment, the input data **1520** comprises commands that configure the processing units to implement

the stages of the graphics processing pipeline **1500** and geometric primitives (e.g., points, lines, triangles, quads, triangle strips or fans, etc.) to be processed by the stages. The output data **1502** may comprise pixel data (e.g., color data) that is copied into a frame buffer or other type of surface data structure in a memory.

[0140] The data assembly **1504** stage receives the input data **1520** that specifies vertex data for high-order surfaces, primitives, or the like. The data assembly **1504** stage collects the vertex data in a temporary storage or queue, such as by receiving a command from the host processor that includes a pointer to a buffer in memory and reading the vertex data from the buffer. The vertex data is then transmitted to the vertex shading **1506** stage for processing.

[0141] The vertex shading **1506** stage processes vertex data by performing a set of operations (e.g., a vertex shader or a program) once for each of the vertices. Vertices may be, e.g., specified as a 4-coordinate vector (e.g., $\langle x, y, z, w \rangle$) associated with one or more vertex attributes (e.g., color, texture coordinates, surface normal, etc.). The vertex shading **1506** stage may manipulate individual vertex attributes such as position, color, texture coordinates, and the like. In other words, the vertex shading **1506** stage performs operations on the vertex coordinates or other vertex attributes associated with a vertex. Such operations commonly including lighting operations (e.g., modifying color attributes for a vertex) and transformation operations (e.g., modifying the coordinate space for a vertex). For example, vertices may be specified using coordinates in an object-coordinate space, which are transformed by multiplying the coordinates by a matrix that translates the coordinates from the object-coordinate space into a world space or a normalized-device-coordinate (NCD) space. The vertex shading **1506** stage generates transformed vertex data that is transmitted to the primitive assembly **1508** stage.

[0142] The primitive assembly **1508** stage collects vertices output by the vertex shading **1506** stage and groups the vertices into geometric primitives for processing by the geometry shading **1510** stage. For example, the primitive assembly **1508** stage may be configured to group every three consecutive vertices as a geometric primitive (e.g., a triangle) for transmission to the geometry shading **1510** stage. In some embodiments, specific vertices may be reused for consecutive geometric primitives (e.g., two consecutive triangles in a triangle strip may share two vertices). The primitive assembly **1508** stage transmits geometric primitives (e.g., a collection of associated vertices) to the geometry shading **1510** stage.

[0143] The geometry shading **1510** stage processes geometric primitives by performing a set of operations (e.g., a geometry shader or program) on the geometric primitives. Tessellation operations may generate one or more geometric primitives from each geometric primitive. In other words, the geometry shading **1510** stage may subdivide each geometric primitive into a finer mesh of two or more geometric primitives for processing by the rest of the graphics processing pipeline **1500**. The geometry shading **1510** stage transmits geometric primitives to the viewport SCC **1512** stage.

[0144] In an embodiment, the graphics processing pipeline **1500** may operate within a streaming multiprocessor and the vertex shading **1506** stage, the primitive assembly **1508** stage, the geometry shading **1510** stage, the fragment shading **1516** stage, and/or hardware/software associated there-

with, may sequentially perform processing operations. Once the sequential processing operations are complete, in an embodiment, the viewport SCC **1512** stage may utilize the data. In an embodiment, primitive data processed by one or more of the stages in the graphics processing pipeline **1500** may be written to a cache (e.g. L1 cache, a vertex cache, etc.). In this case, in an embodiment, the viewport SCC **1512** stage may access the data in the cache. In an embodiment, the viewport SCC **1512** stage and the rasterization **1514** stage are implemented as fixed function circuitry.

[0145] The viewport SCC **1512** stage performs viewport scaling, culling, and clipping of the geometric primitives. Each surface being rendered to is associated with an abstract camera position. The camera position represents a location of a viewer looking at the scene and defines a viewing frustum that encloses the objects of the scene. The viewing frustum may include a viewing plane, a rear plane, and four clipping planes. Any geometric primitive entirely outside of the viewing frustum may be culled (e.g., discarded) because the geometric primitive will not contribute to the final rendered scene. Any geometric primitive that is partially inside the viewing frustum and partially outside the viewing frustum may be clipped (e.g., transformed into a new geometric primitive that is enclosed within the viewing frustum. Furthermore, geometric primitives may each be scaled based on a depth of the viewing frustum. All potentially visible geometric primitives are then transmitted to the rasterization **1514** stage.

[0146] The rasterization **1514** stage converts the 3D geometric primitives into 2D fragments (e.g. capable of being utilized for display, etc.). The rasterization **1514** stage may be configured to utilize the vertices of the geometric primitives to setup a set of plane equations from which various attributes can be interpolated. The rasterization **1514** stage may also compute a coverage mask for a plurality of pixels that indicates whether one or more sample locations for the pixel intercept the geometric primitive. In an embodiment, z-testing may also be performed to determine if the geometric primitive is occluded by other geometric primitives that have already been rasterized. The rasterization **1514** stage generates fragment data (e.g., interpolated vertex attributes associated with a particular sample location for each covered pixel) that are transmitted to the fragment shading **1516** stage.

[0147] The fragment shading **1516** stage processes fragment data by performing a set of operations (e.g., a fragment shader or a program) on each of the fragments. The fragment shading **1516** stage may generate pixel data (e.g., color values) for the fragment such as by performing lighting operations or sampling texture maps using interpolated texture coordinates for the fragment. The fragment shading **1516** stage generates pixel data that is transmitted to the raster operations **1518** stage.

[0148] The raster operations **1518** stage may perform various operations on the pixel data such as performing alpha tests, stencil tests, and blending the pixel data with other pixel data corresponding to other fragments associated with the pixel. When the raster operations **1518** stage has finished processing the pixel data (e.g., the output data **1502**), the pixel data may be written to a render target such as a frame buffer, a color buffer, or the like.

[0149] It will be appreciated that one or more additional stages may be included in the graphics processing pipeline **1500** in addition to or in lieu of one or more of the stages

described above. Various implementations of the abstract graphics processing pipeline may implement different stages. Furthermore, one or more of the stages described above may be excluded from the graphics processing pipeline in some embodiments (such as the geometry shading **1510** stage). Other types of graphics processing pipelines are contemplated as being within the scope of the present disclosure. Furthermore, any of the stages of the graphics processing pipeline **1500** may be implemented by one or more dedicated hardware units within a graphics processor such as parallel processing unit **902**. Other stages of the graphics processing pipeline **1500** may be implemented by programmable hardware units such as the streaming multiprocessor **1200** of the parallel processing unit **902**.

[0150] The graphics processing pipeline **1500** may be implemented via an application executed by a host processor, such as a CPU. In an embodiment, a device driver may implement an application programming interface (API) that defines various functions that can be utilized by an application in order to generate graphical data for display. The device driver is a software program that includes a plurality of instructions that control the operation of the parallel processing unit **902**. The API provides an abstraction for a programmer that lets a programmer utilize specialized graphics hardware, such as the parallel processing unit **902**, to generate the graphical data without requiring the programmer to utilize the specific instruction set for the parallel processing unit **902**. The application may include an API call that is routed to the device driver for the parallel processing unit **902**. The device driver interprets the API call and performs various operations to respond to the API call. In some instances, the device driver may perform operations by executing instructions on the CPU. In other instances, the device driver may perform operations, at least in part, by launching operations on the parallel processing unit **902** utilizing an input/output interface between the CPU and the parallel processing unit **902**. In an embodiment, the device driver is configured to implement the graphics processing pipeline **1500** utilizing the hardware of the parallel processing unit **902**.

[0151] Various programs may be executed within the parallel processing unit **902** in order to implement the various stages of the graphics processing pipeline **1500**. For example, the device driver may launch a kernel on the parallel processing unit **902** to perform the vertex shading **1506** stage on one streaming multiprocessor **1200** (or multiple streaming multiprocessor **1200** modules). The device driver (or the initial kernel executed by the parallel processing unit **902**) may also launch other kernels on the parallel processing unit **902** to perform other stages of the graphics processing pipeline **1500**, such as the geometry shading **1510** stage and the fragment shading **1516** stage. In addition, some of the stages of the graphics processing pipeline **1500** may be implemented on fixed unit hardware such as a rasterizer or a data assembler implemented within the parallel processing unit **902**. It will be appreciated that results from one kernel may be processed by one or more intervening fixed function hardware units before being processed by a subsequent kernel on a streaming multiprocessor **1200**.

[0152] FIG. 16 depicts an exemplary data center **1600**, in accordance with at least one embodiment. The data center **1600** includes, without limitation, a data center infrastructure layer **1602**, a framework layer **1604**, software layer **1606**, and an application layer **1608**. The data center **1600**

may execute many deep learning workloads concurrently, and may utilize many graphics processing units and/or microprocessors to do so.

[0153] The data center infrastructure layer **1602** may include a resource orchestrator **1610**, grouped computing resources **1612**, and node computing resources (“node C.R.s”) Node C.R. **1614a**, Node C.R. **1614b**, Node C.R. **1614c**, . . . node C.R. **N**), where “N” represents any whole, positive integer. The node C.R.s may include, but are not limited to, any number of central processing units (“CPUs”) or other processors (including accelerators, field programmable gate arrays (“FPGAs”), graphics processing units, etc.), memory devices (e.g., dynamic read-only memory), storage devices (e.g., solid state or disk drives), network input/output (“NW I/O”) devices, network switches, virtual machines (“VMs”), power modules, and cooling modules, etc. One or more node C.R.s from among node C.R.s may be a server having one or more of above-mentioned computing resources.

[0154] The grouped computing resources **1612** may include separate groupings of node C.R.s housed within one or more racks (not shown), or many racks housed in data centers at various geographical locations (also not shown). Separate groupings of node C.R.s within grouped computing resources **1612** may include grouped compute, network, memory or storage resources that may be configured or allocated to support one or more workloads. Several node C.R.s including CPUs or processors may grouped within one or more racks to provide compute resources to support one or more workloads. One or more racks may also include any number of power modules, cooling modules, and network switches, in any combination.

[0155] The resource orchestrator **1610** may configure or otherwise control one or more node C.R.s and/or grouped computing resources **1612**. The resource orchestrator **1610** may include a software design infrastructure (“SDI”) management entity for data center **1600**. The resource orchestrator **1610** may include hardware, software or some combination thereof.

[0156] A framework layer **1604** includes, without limitation, a job scheduler **1616**, a configuration manager **1618**, a resource manager **1620**, and a distributed file system **1622**. The framework layer **1604** may include a framework to support software **1624** of software layer **1606** and/or one or more application(s) **1626** of application layer **220**. The software **1624** or application(s) **1626** may respectively include web-based service software or applications, such as those provided by Amazon Web Services, Google Cloud and Microsoft Azure.

[0157] The framework layer **1604** may be, but is not limited to, a type of free and open-source software web application framework such as Apache Spark™ (hereinafter “Spark”) that may utilize a distributed file system **1622** for large-scale data processing (e.g., “big data”). The job scheduler **1616** may include a Spark driver to facilitate scheduling of workloads supported by various layers of data center **1600**.

[0158] The configuration manager **1618** may be capable of configuring different layers such as software layer **1606** and framework layer **1604**, including Spark and distributed file system **1622** for supporting large-scale data processing. The resource manager **1620** may be capable of managing clustered or grouped computing resources mapped to or allocated for support of distributed file system **1622** and distributed file system **1622**. The clustered or grouped

computing resources may include grouped computing resources **1612** at data center infrastructure layer **1602**. The resource manager **1620** may coordinate with resource orchestrator **1610** to manage these mapped or allocated computing resources.

[0159] The software **1624** included in software layer **1606** may include software used by at least portions of node C.R.s, grouped computing resources **1612**, and/or distributed file system **1622** of framework layer **1604**. One or more types of software may include, but are not limited to, Internet web page search software, e-mail virus scan software, database software, and streaming video content software.

[0160] The application(s) **1626** included in application layer **1608** may include one or more types of applications used by at least portions of node C.R.s, grouped computing resources **1612**, and/or distributed file system **1622** of framework layer **1604**. In at least one or more types of applications may include, without limitation, CUDA applications, 5G network applications, artificial intelligence application, data center applications, and/or variations thereof.

[0161] Any of configuration manager **1618**, resource manager **1620**, and resource orchestrator **1610** may implement any number and type of self-modifying actions based on any amount and type of data acquired in any technically feasible fashion. Self-modifying actions may relieve a data center operator of data center **1600** from making possibly bad configuration decisions and possibly avoiding underutilized and/or poor performing portions of a data center.

[0162] FIG. 17 depicts exemplary scenarios for use of a circuit system **1702** in accordance with some embodiments. A circuit system **1702** may be utilized in a computing system **1704**, a vehicle **1706**, and a robot **1708**, to name just a few examples. The circuit system **1702** may comprise one or more graphics processing unit executing instructions of a deep learning workload and providing results to a machine memory, for example.

LISTING OF DRAWING ELEMENTS

[0163]	202 multiply units	[0187]	416 multiplier
[0164]	204 mantissa multiplier	[0188]	418 multiplier
[0165]	206 adder	[0189]	420 adder
[0166]	208 XOR logic	[0190]	422 shifter and truncater
[0167]	210 maximum selector	[0191]	424 adder
[0168]	212 shifter	[0192]	502 multiply units
[0169]	214 2's complementer	[0193]	504 adder
[0170]	216 adder	[0194]	506 mantissa multiplier
[0171]	218 normalizer	[0195]	508 XOR logic
[0172]	302 multiply unit	[0196]	510 2's complementer
[0173]	304 adder	[0197]	512 shifter
[0174]	306 mantissa multiplier	[0198]	514 adder
[0175]	308 XOR logic	[0199]	516 multiplier
[0176]	310 rounder	[0200]	518 multiplier
[0177]	312 2's complementer	[0201]	520 adder
[0178]	314 shifter and rounder	[0202]	522 aligner
[0179]	316 adder	[0203]	524 adder
[0180]	402 multiply units	[0204]	526
[0181]	404 adder	[0205]	602 multiply-accumulate units
[0182]	406 mantissa multiplier	[0206]	604 adder
[0183]	408 XOR logic	[0207]	606 look-up table
[0184]	410 2's complementer	[0208]	608 shifter
[0185]	412 shifter and truncater	[0209]	610 truncater
[0186]	414 adder	[0210]	612 2's complementer
		[0211]	614 ADD
		[0212]	616 adder tree
		[0213]	700 neural network processor
		[0214]	702 processing elements
		[0215]	704 global buffer
		[0216]	706 controller
		[0217]	708 network-on-a-package router
		[0218]	800 processing element
		[0219]	802 vector multiply-accumulate units
		[0220]	804 activation buffer
		[0221]	806 router
		[0222]	808 controller
		[0223]	810 accumulation memory buffer
		[0224]	812 weight buffer
		[0225]	814 post-processor
		[0226]	902 parallel processing unit
		[0227]	904 I/O unit
		[0228]	906 front-end unit
		[0229]	908 scheduler unit
		[0230]	910 work distribution unit
		[0231]	912 hub
		[0232]	914 crossbar
		[0233]	916 NVLink
		[0234]	918 interconnect
		[0235]	920 memory
		[0236]	1000 general processing cluster
		[0237]	1002 pipeline manager
		[0238]	1004 pre-raster operations unit
		[0239]	1006 raster engine
		[0240]	1008 work distribution crossbar
		[0241]	1010 memory management unit
		[0242]	1012 data processing cluster
		[0243]	1014 primitive engine
		[0244]	1016 M-pipe controller
		[0245]	1100 memory partition unit
		[0246]	1102 raster operations unit
		[0247]	1104 level two cache
		[0248]	1106 memory interface
		[0249]	1200 streaming multiprocessor
		[0250]	1202 instruction cache

[0251]	1204 scheduler unit
[0252]	1206 register file
[0253]	1208 core
[0254]	1210 special function unit
[0255]	1212 load/store unit
[0256]	1214 interconnect network
[0257]	1216 shared memory/L1 cache
[0258]	1218 dispatch
[0259]	1300 processing system
[0260]	1302 central processing unit
[0261]	1304 switch
[0262]	1306 parallel processing module
[0263]	1400 exemplary processing system
[0264]	1402 communications bus
[0265]	1404 main memory
[0266]	1406 input devices
[0267]	1408 display devices
[0268]	1410 network interface
[0269]	1500 graphics processing pipeline
[0270]	1502 output data
[0271]	1504 data assembly
[0272]	1506 vertex shading
[0273]	1508 primitive assembly
[0274]	1510 geometry shading
[0275]	1512 viewport SCC
[0276]	1514 rasterization
[0277]	1516 fragment shading
[0278]	1518 raster operations
[0279]	1520 input data
[0280]	1600 data center
[0281]	1602 data center infrastructure layer
[0282]	1604 framework layer
[0283]	1606 software layer
[0284]	1608 application layer
[0285]	1610 resource orchestrator
[0286]	1612 grouped computing resources
[0287]	1614a node C.R.
[0288]	1614b node C.R.
[0289]	1614c node C.R.
[0290]	1616 job scheduler
[0291]	1618 configuration manager
[0292]	1620 resource manager
[0293]	1622 distributed file system
[0294]	1624 software
[0295]	1626 application(s)
[0296]	1702 circuit system
[0297]	1704 computing system
[0298]	1706 vehicle
[0299]	1708 robot

[0300] Various functional operations described herein may be implemented in logic that is referred to using a noun or noun phrase reflecting said operation or function. For example, an association operation may be carried out by an “associator” or “correlator”. Likewise, switching may be carried out by a “switch”, selection by a “selector”, and so on. “Logic” refers to machine memory circuits and non-transitory machine readable media comprising machine-executable instructions (software and firmware), and/or circuitry (hardware) which by way of its material and/or material-energy configuration comprises control and/or procedural signals, and/or settings and values (such as resistance, impedance, capacitance, inductance, current/voltage ratings, etc.), that may be applied to influence the operation of a device. Magnetic media, electronic circuits, electrical

and optical memory (both volatile and nonvolatile), and firmware are examples of logic. Logic specifically excludes pure signals or software per se (however does not exclude machine memories comprising software and thereby forming configurations of matter).

[0301] Within this disclosure, different entities (which may variously be referred to as “units,” “circuits,” other components, etc.) may be described or claimed as “configured” to perform one or more tasks or operations. This formulation—[entity] configured to [perform one or more tasks]—is used herein to refer to structure (i.e., something physical, such as an electronic circuit). More specifically, this formulation is used to indicate that this structure is arranged to perform the one or more tasks during operation. A structure can be said to be “configured to” perform some task even if the structure is not currently being operated. A “credit distribution circuit configured to distribute credits to a plurality of processor cores” is intended to cover, for example, an integrated circuit that has circuitry that performs this function during operation, even if the integrated circuit in question is not currently being used (e.g., a power supply is not connected to it). Thus, an entity described or recited as “configured to” perform some task refers to something physical, such as a device, circuit, memory storing program instructions executable to implement the task, etc. This phrase is not used herein to refer to something intangible.

[0302] The term “configured to” is not intended to mean “configurable to.” An unprogrammed FPGA, for example, would not be considered to be “configured to” perform some specific function, although it may be “configurable to” perform that function after programming.

[0303] Reciting in the appended claims that a structure is “configured to” perform one or more tasks is expressly intended not to invoke 35 U.S.C. § 112(f) for that claim element. Accordingly, claims in this application that do not otherwise include the “means for” [performing a function] construct should not be interpreted under 35 U.S.C. § 112(f).

[0304] As used herein, the term “based on” is used to describe one or more factors that affect a determination. This term does not foreclose the possibility that additional factors may affect the determination. That is, a determination may be solely based on specified factors or based on the specified factors as well as other, unspecified factors. Consider the phrase “determine A based on B.” This phrase specifies that B is a factor that is used to determine A or that affects the determination of A. This phrase does not foreclose that the determination of A may also be based on some other factor, such as C. This phrase is also intended to cover an embodiment in which A is determined based solely on B. As used herein, the phrase “based on” is synonymous with the phrase “based at least in part on.”

[0305] As used herein, the phrase “in response to” describes one or more factors that trigger an effect. This phrase does not foreclose the possibility that additional factors may affect or otherwise trigger the effect. That is, an effect may be solely in response to those factors, or may be in response to the specified factors as well as other, unspecified factors. Consider the phrase “perform A in response to B.” This phrase specifies that B is a factor that triggers the performance of A. This phrase does not foreclose that performing A may also be in response to some other factor, such as C. This phrase is also intended to cover an embodiment in which A is performed solely in response to B.

[0306] As used herein, the terms “first,” “second,” etc. are used as labels for nouns that they precede, and do not imply any type of ordering (e.g., spatial, temporal, logical, etc.), unless stated otherwise. For example, in a register file having eight registers, the terms “first register” and “second register” can be used to refer to any two of the eight registers, and not, for example, just logical registers 0 and 1.

[0307] When used in the claims, the term “or” is used as an inclusive or and not as an exclusive or. For example, the phrase “at least one of x, y, or z” means any one of x, y, and z, as well as any combination thereof.

[0308] As used herein, a recitation of “and/or” with respect to two or more elements should be interpreted to mean only one element, or a combination of elements. For example, “element A, element B, and/or element C” may include only element A, only element B, only element C, element A and element B, element A and element C, element B and element C, or elements A, B, and C. In addition, “at least one of element A or element B” may include at least one of element A, at least one of element B, or at least one of element A and at least one of element B. Further, “at least one of element A and element B” may include at least one of element A, at least one of element B, or at least one of element A and at least one of element B.

[0309] The subject matter of the present disclosure is described with specificity herein to meet statutory requirements. However, the description itself is not intended to limit the scope of this disclosure. Rather, the inventors have contemplated that the claimed subject matter might also be embodied in other ways, to include different steps or combinations of steps similar to the ones described in this document, in conjunction with other present or future technologies. Moreover, although the terms “step” and/or “block” may be used herein to connote different elements of methods employed, the terms should not be interpreted as implying any particular order among or between various steps herein disclosed unless and except when the order of individual steps is explicitly described.

[0310] Having thus described illustrative embodiments in detail, it will be apparent that modifications and variations are possible without departing from the scope of the invention as claimed. The scope of inventive subject matter is not limited to the depicted embodiments but is rather set forth in the following Claims.

What is claimed is:

1. A computer processor comprising:
 - a plurality of multiplier units each configured to multiply two low-precision floating-point VS-QUANT operands to generate a first INT result;
 - an adder to reduce the first INT results to a second INT result; and
 - logic to multiply the second INT result by a low-precision floating-point scale factor.
2. The computer processor of claim 1, the multiplier unit comprising:
 - an exponent adder to generate an exponent sum;
 - a mantissa multiplier to generate a mantissa product; and
 - logic configured to combine the exponent sum, the mantissa product, and a sign bit to generate the first INT result.
3. The computer processor of claim 2, the multiplier unit further configured to add an exponent bias to the exponent sum.

4. A computer processor comprising:
 - a multiplier unit configured to add two low-precision floating point VS-Quant LOG operands into a first sum;
 - logic to convert a portion of the first sum into an INT result; and
 - logic to convert the INT result to an INT product of the floating-point operands.
5. A computer processor floating-point multiplier unit comprising:
 - an adder configured to add two exponent values of low-precision floating point VS-Quant LOG operands into an exponent sum;
 - a lookup table configured to convert a portion of the exponent sum into a mantissa product;
 - logic configured to XOR two sign bits of the low-precision floating point VS-Quant LOG operands into a sign bit result;
 - logic configured to:
 - complement the mantissa product based on the sign bit into a partial result; and
 - shift the partial result based on the exponent sum to generate an INT result.
6. A computer processor comprising:
 - a plurality of multiplier units each configured to multiply two VS -Quant FP4 floating-point operands to generate a first INT result;
 - an adder to reduce the first INT results to a second INT result; and
 - logic to multiply the second INT result by a low-precision floating-point scale factor.
7. The computer processor of claim 6, the multiplier unit comprising:
 - an exponent adder to generate an exponent sum;
 - a mantissa multiplier to generate a mantissa product; and
 - logic configured to combine the exponent sum, the mantissa product, and a sign bit to generate the first INT result.
8. The computer processor of claim 7, wherein the logic to combine the exponent sum, the mantissa product, and the sign bit to generate the first INT result consists of complementing, shifting, and truncating logic.
9. The computer processor of claim 8, wherein the complementing, shifting, and truncating logic are arranged in a pipeline.
10. The computer processor of claim 6, wherein the logic to multiply the second INT result by the low-precision floating-point scale factor comprises logic to multiply a product of mantissa scale factors for activation values and for weight values with the first INT result to generate an intermediate value.
11. The computer processor of claim 10, wherein the logic to multiply the second INT result by the low-precision floating-point scale factor further comprises:
 - logic to shift the intermediate value by a sum of exponent scale factors for the activation values and the weight values.
12. The computer processor of claim 10, wherein the logic to multiply the second INT result by the low-precision floating-point scale factor further comprises logic to align an exponent of a floating-point accumulator operand and a sum of exponent scale factors for the activation values and the weight values.
13. A computer processor comprising:
 - a multiplier unit configured to add two VS-Quant LOG4 floating-point operands into a first sum;

- logic to convert a portion of the first sum into an INT result; and
- logic to convert the INT result to an INT product of the floating-point operands.
14. The computer processor of claim 13, wherein the floating-point operands are exponent values.
15. The computer processor of claim 13, the multiplier unit further configured to add an exponent bias to the first sum.
16. The computer processor of claim 13, wherein the logic to convert the INT result to an INT product of the floating-point operands consists of complementing, shifting, and truncating logic.
17. The computer processor of claim 16, wherein the complementing, shifting, and truncating logic are arranged in a pipeline.
18. The computer processor of claim 13, further comprising an adder to add the INT product to a third operand.
19. The computer processor of claim 13, wherein the logic to convert a portion of the first sum into an INT result comprises a lookup table.
20. A computer processor floating-point multiplier unit comprising:
- an adder configured to add two exponent values of VS-Quant FP4 operands into an exponent sum;
 - a multiplier configured to multiply two mantissa values of the VS-Quant FP4 operands into a mantissa product;
 - logic configured to XOR two sign bits of the VS-Quant FP4 operands into a sign bit result;
 - logic configured to:
 - complement the mantissa product based on the sign bit into a partial result; and
 - shift the partial result based on the exponent sum to generate an INT result.
21. The processor floating-point multiplier unit of claim 20, further comprising logic to truncate the INT result.
22. A computer processor floating-point multiplier unit comprising:
- an adder configured to add two exponent values of VS-Quant LOG4 operands into an exponent sum;
 - a lookup table configured to convert a portion of the exponent sum into a mantissa product;
 - logic configured to XOR two sign bits of the VS-Quant LOG4 operands into a sign bit result;
 - logic configured to:
 - complement the mantissa product based on the sign bit into a partial result; and
 - shift the partial result based on the exponent sum to generate an INT result.
23. The processor floating-point multiplier unit of claim 22, further comprising logic to truncate the INT result.
24. The processor floating-point multiplier unit of claim 22, the adder further configured to apply a bias to the addition of the two exponent values.
25. A method for reducing power consumption of floating-point calculations in a computer processor, the method comprising:
- in a plurality of multiplier units:
 - adding two exponents into an exponent sum;
 - multiplying two mantissa values into a mantissa product;
 - XORing two sign bits into a sign bit result;
 - complementing and shifting the mantissa product based on the sign bit into an INT partial product; and
 - adding the INT partial products from the plurality of multiplier units to generate a dot-product sum.
26. The method of claim 25, further comprising:
- multiplying two mantissa scale factors to generate a first scale factor product;
 - multiplying the dot-product sum by the first scale factor product to generate a first partial scaled result; and
 - adding two exponent scale factors to generate a first exponent sum.
27. The method of claim 26, further comprising:
- aligning the first partial scaled result and first exponent sum with other partial scaled results and exponent sums, and partial accumulated FP sum to generate aligned partial scaled results and aligned exponents; and
 - adding the aligned partial scaled results and aligned exponents to generate an accumulated result.
- * * * * *